Внедрение CI/CD с использованием GitLab

Автоматизация сборки, тестирования и деплоя

Author: Aliaksandr Tsimokhau

Agenda

Part 1: Основы CI/CD

Стадии, CI vs CD vs Deployment, метрики, rollback

Part 2: Безопасность и тестирование

Security pipeline, continuous testing

Part 3: Введение в GitLab CI/CD

Архитектура, `.gitlab-ci.yml`, первый pipeline

Part 4: Структура `.gitlab-ci.yml`

Stages, jobs, dependencies, переменные, артефакты

Part 5: Rules и условия запуска

`rules: if / changes / exists / source`

Part 6: Расширенные возможности

YAML anchors, extends, triggers, matrix

Part 7: Includes и Cache vs Artifacts

`include:local/project/remote`, кэш, артефакты

Part 8: Реальные паттерны GitLab CI

Monorepo, microservices, dynamic pipelines, MR-first, CI/CD Components

Part 9: Best Practices и лабы

Best practices и лабораторные

Part 1: Основы CI/CD

Что такое pipeline и из чего он состоит

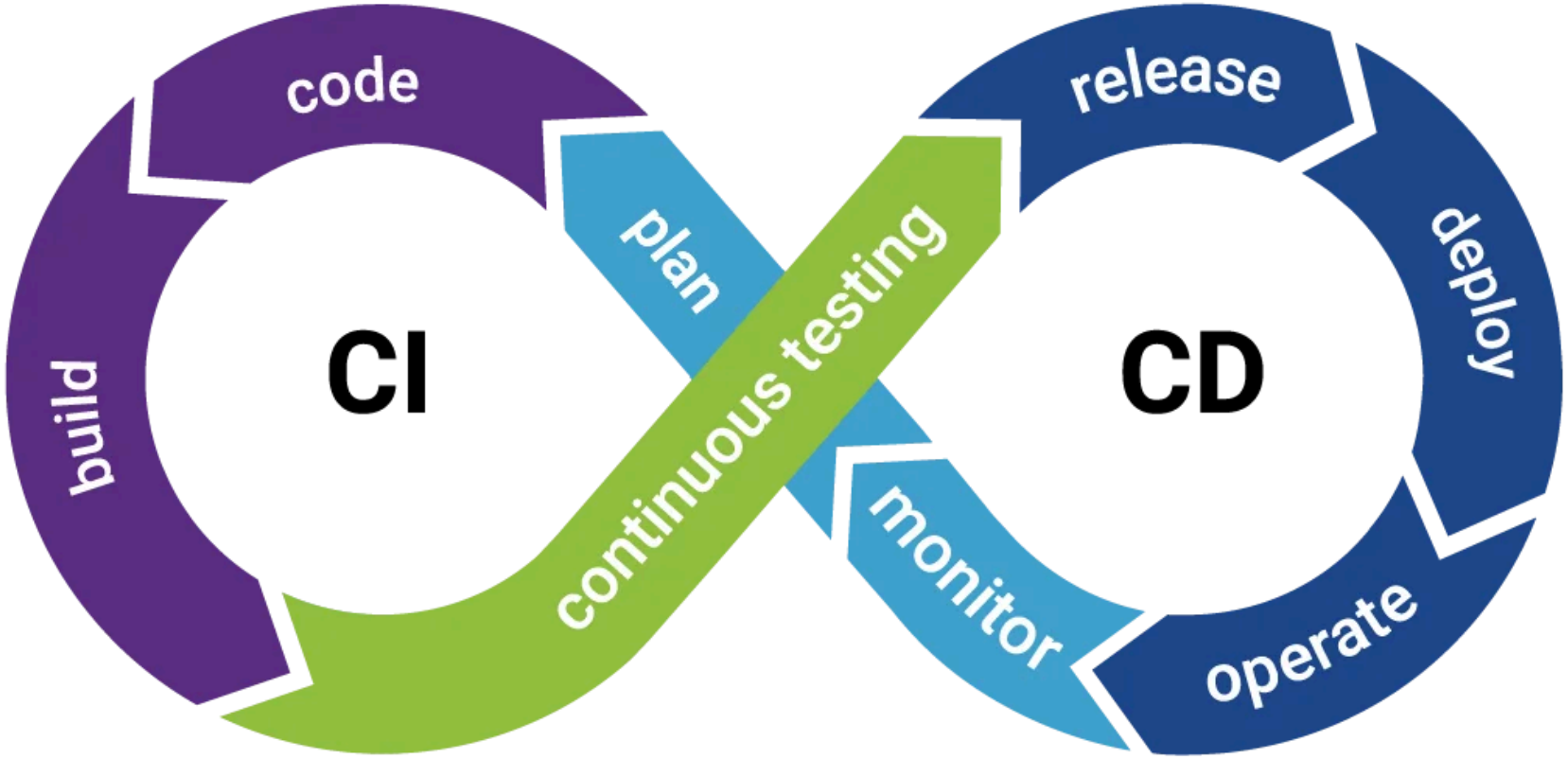
Классическая схема CI/CD



Основные этапы CI/CD Pipeline

- **Source stage**: изменения в коде или расписание запускают пайплайн
- **Build stage**: сборка приложения и зависимостей
- **Testing stage**: автоматизированное тестирование качества и поведения кода
- **Deploy**: развертывание проверенного приложения в нужной среде

Основы непрерывной интеграции и непрерывной доставки



Непрерывная интеграция (CI)

Основные шаги CI:

- **Коммит изменений**: разработчики коммитят изменения в общий репозиторий
- **Автоматическая сборка**: система CI автоматически запускает сборку проекта
- **Автоматическое тестирование**: после сборки запускаются автоматические тесты для проверки корректности изменений
- **Отчет о результатах**: система CI предоставляет отчет о результатах сборки и тестирования

Непрерывная доставка (CD)

Основные шаги CD:

- **Подготовка артефактов**: после успешного прохождения CI готовим артефакты для деплоя
- **Развертывание в staging**: артефакты автоматически разворачиваются в staging
- **Автоматическое тестирование**: в staging запускаются интеграционные / smoke тесты
- **Ручное или автоматическое развертывание в production**

Continuous Integration vs Delivery vs Deployment



CI/CD Pipeline — high-level stages

The main high-level stages of a standard CI/CD pipeline:

- **Source stage** — push / schedule / trigger
- **Build stage** — компиляция, упаковка артефакта
- **Testing stage** — unit / integration / e2e тесты
- **Deployment stage** — staging → production

Компоненты CI/CD pipeline



Stage One — Source

- Pipeline запускается при изменении в репозитории
- Может быть запущен по расписанию или другим pipeline
- Старт автоматизированного CI/CD процесса

Stage Two — Build

- Исходный код и зависимости компилируются или собираются
- Падение этой стадии — фундаментальная проблема конфигурации
- Только успешный билд идёт дальше

Stage Three — Testing

- Автоматические тесты валидируют качество и поведение кода
- Длительность зависит от размера проекта
- Найденные проблемы должны быть устранены до деплоя

Stage Four — Deployment

- Успешно протестированное ПО разворачивается в целевые среды
- Несколько окружений — staging, production
- Быстрая и надёжная доставка для agile-команд

CI/CD Pipeline: Advanced Benefits

- **Efficient development**: маленькие частые изменения упрощают тестирование и фиксы
- **Rapid adaptation**: быстро реагируем на новые требования
- **Supports experimentation**: безопасные эксперименты с откатами
- **Improved maintenance**: непрерывный поток ускоряет фиксы багов и стабильность
- **Higher code quality**: ранние ошибки = более надёжный продукт

Potential Drawbacks of CI/CD Pipelines

- **High learning curve**
  - Требует устоявшихся инструментов и зрелых процессов
  - Команда должна обучиться и перестроить процессы
- **Value depends on project goals**
  - Максимум пользы — частые релизы и обновления
  - Не оправдывает оверхеда при редких изменениях

Drawbacks — tools, security, cost

Tool compatibility and integration complexity

- Управление несколькими инструментами может быть сложным
- Выбирайте хорошо интегрированные решения
- Обеспечьте знакомство команды с выбранными инструментами

Security risks in automated deployments

- Автоматизация может раскрыть чувствительные данные или привести к уязвимостям
- Используйте безопасные практики, шифруйте секреты, внедряйте контроль доступа
- Регулярно запускайте автоматизированные проверки безопасности

ROI of CI/CD Pipelines

- **Снижение операционных расходов** через автоматизацию рутины
- **Ускорение time-to-market** за счёт быстрых и надёжных релизов
- **Повышение удовлетворённости клиентов** частыми и стабильными обновлениями

Best Practices for CI/CD Pipelines

- **Commit code frequently** — маленькие коммиты, частые
- **Automate everything possible** — никакой ручной работы
- **Use parallel testing** — параллельные jobs для скорости
- **Monitor and log** — observability на каждом шаге

Security Considerations for CI/CD Pipelines

- **Follow secure coding practices** — security checks с первой строки кода
- **Manage secrets securely** — никаких токенов в коде
- **Automate security testing** — SAST/DAST в pipeline

CI/CD Pipeline Metrics to Track

- **Build Success Rate** — % успешных билдов
- **Mean Time to Recovery (MTTR)** — время восстановления после падения
- **Deployment Frequency** — частота деплоев в прод
- **Lead Time for Changes** — время от коммита до деплоя

Part 2: Стратегии Rollback и безопасность

Что делать, когда деплой пошёл не так

Rollback Strategies in CI/CD

- **Автоматические процедуры отката** минимизируют риски и downtime
- Инструменты: Git, Jenkins, Terraform, GitLab CI, ArgoCD, Helm
- Основные подходы к rollback:
  - **Red-Black (Blue-Green) Deployment**
  - **Canary Releases**
  - **Rolling Back Database Migrations**
  - **Snapshot Rollback**

Red-Black (Blue-Green) Deployment

- Держим две production-среды: "Red" (текущая) и "Black" (новая)
- Деплоим и тестируем новую версию в "Black"
- Переключаем трафик на "Black" после успешного тестирования
- Если проблемы — возвращаем трафик на "Red"

Canary Releases

- Постепенно раскатываем новую версию на маленькую часть пользователей / серверов
- Мониторим проблемы перед полным rollout
- Stop / reverse при детекции проблем

Rolling Back Database Migrations

- Готовим скрипты для отката изменений схемы БД
- Критично для безопасного восстановления предыдущих версий

Snapshot Rollback

- Делаем snapshot системы или компонента перед деплоем
- Восстанавливаемся из snapshot при необходимости

Continuous Testing and Security Integration

- **Continuous Testing**: автоматические тесты на всех этапах
- **Security Integration**: security checks встроены в каждую фазу CI/CD
- **Key Tools**: Selenium, TestNG, Veracode, SonarQube, Trivy, OWASP ZAP

Part 3: Введение в GitLab CI/CD

Архитектура и первый pipeline

What Is GitLab CI/CD?

- **GitLab CI/CD** — инструмент для continuous практик:
  - Continuous Integration (CI)
  - Continuous Delivery (CD)
  - Continuous Deployment (CD)
- Раннее обнаружение ошибок в SDLC
- Код в проде соответствует compliance и coding standards

GitLab — больше, чем «git + CI»

DevSecOps platform: одна установка покрывает весь SDLC.

10 этапов SDLC в одной платформе:

- Manage · Plan · Create
- **Verify** (CI) · Package · Secure
- **Release** (CD) · Configure
- Monitor · Govern

Что входит «из коробки»:

- Git hosting (Gitaly)
- Issues, MR, wiki
- CI/CD + Runners
- Container & Package Registry
- Security scanners (SAST/DAST/SCA)
- Pages · Environments · Feature flags

> 💡 Конкуренты — это **набор интегрированных tools**: GitHub + Actions + Dependabot + ghcr + Pages. У GitLab всё это — один продукт.

Архитектура — high-level



Application tier — детальнее

NGINX

- TLS termination
- Routing: `/api`, `/-/`, репозитории
- В k8s: Ingress controller

Workhorse

- Go, ставится перед Puma
- Обрабатывает «толстые» запросы:
  - `git clone/push`
  - file uploads (large MIME)
  - websocket (live UI, terminals)

Puma

- Rails app server (Ruby)
- Web UI, REST, GraphQL
- N workers × M threads
- CPU/memory hungry

Sidekiq

- Background job processor
- Очереди: `default`, `mailers`, `pipeline_processing`, ...
- Critical для async всего: webhooks, scans, deletion

Data tier — где живут данные

Сервис	Что хранит	Особенности
PostgreSQL	users, projects, issues, MRs, pipelines, audit	primary store · HA через Patroni/Pacemaker
Redis	sessions, cache, Sidekiq queues	критичный · падает → GitLab down
Object Storage	CI artifacts, LFS, registry blobs, packages	S3-compatible · самый большой по объёму
Gitaly disk	bare git repos (`.git/`)	filesystem · Praefect для HA
Elasticsearch *(optional)*	advanced search (code, issues)	Premium+ feature

Gitaly — почему отдельный сервис

До 2018: Rails ходил напрямую в filesystem, fork'ал `git` процессы.

Проблемы:

- fork на каждый clone → 1000 reqs/sec не выживут
- Rails worker блокировался на git операциях минутами
- нет шардинга — все репы на одной NFS

Gitaly решение:

Gitaly = gRPC сервер:

- bare repos на локальной файловой системе
- RPC методы: `ReceivePack`, `GetCommit`, `TreeEntry`, ...
- один процесс — много connections
- internal repo cache

Praefect — Gitaly cluster manager:

- routing `repo_X → Gitaly node N`
- async replication между nodes
- automatic failover на read failure
- read-from-replica

GitLab Runner — execution model



Варианты развёртывания

SaaS / Managed

- **gitlab.com** — самый простой старт, shared runners, миллион проектов
- **GitLab Dedicated** — single-tenant SaaS, изолированный инстанс в AWS

Self-managed

- **Omnibus** — `.deb` / `.rpm` пакет, всё в одном
- **GitLab Helm chart** — для Kubernetes
- **GitLab Operator** — для k8s + OpenShift с CRD

Когда что выбирать

- gitlab.com → стартапы, OSS, MVP
- Dedicated → enterprise compliance (data residency)
- Omnibus → on-premise VM, не используете k8s
- Helm → у вас уже k8s, нужна горизонтальная scale
- Operator → declarative управление, RHOCP

Поддержка: все варианты получают одинаковые feature releases (раз в месяц)

Editions & Tiers

	Free	Premium	Ultimate
Git, Issues, MR, Wiki	✅	✅	✅
CI/CD + Runners	✅	✅	✅
Container & Package Registry	✅	✅	✅
Code reviews + approvals	basic	multiple approval rules	+ license compliance
Security scanners	secret detection, SAST (шаблон)	+ результаты SAST в MR (проверь тариф)	+ DAST, SCA, fuzz testing
Advanced search (ES)	❌	✅	✅
Compliance frameworks	❌	❌	✅
Audit events	❌	basic	full + streaming
Service Desk / SLA	❌	✅	✅
GitLab Duo (AI)	❌	add-on	add-on

Editions:

- **CE (Community Edition)** = open source, MIT
- **EE (Enterprise Edition)** = same code + закрытые feature flags для Premium/Ultimate

GitLab CI pipeline



Пример CI/CD pipeline для GitLab

```yaml
stages:
  - build
  - test
  - deploy

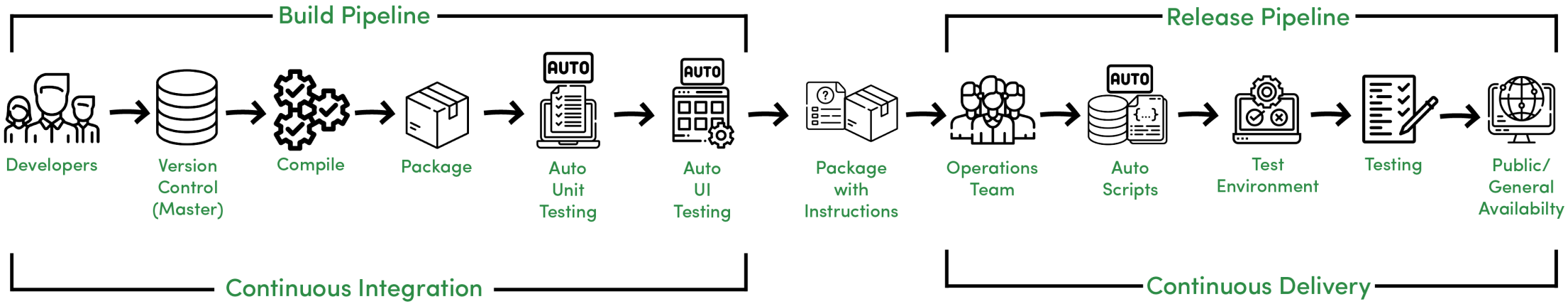
build:
  stage: build
  script:
    - docker build -t my-app:latest .  # собираем образ
    - docker push registry.example.com/my-app:latest

test:
  stage: test
  script:
    - docker run --rm my-app:latest pytest  # тесты в образе

deploy:
  stage: deploy
  script:
    - kubectl apply -f k8s/deployment.yaml
  environment:
    name: production          # видно в Operations → Environments
    url: https://example.com
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'  # только main
```

- **3 стадии:** `build → test → deploy` — test стартует лишь после успешного build
- **`environment`** связывает job с окружением в GitLab UI — история деплоев, rollback
- **`rules: - if:`** — современный способ управления запуском; `only/except` устарели
- **Изоляция:** между jobs нет общей ФС — используйте `artifacts` или `cache`
- **Реальные добавки:** `image:`, `services: [docker:dind]`, `cache:` для зависимостей
- **Готча:** push в registry требует `CI_REGISTRY_USER` / `CI_REGISTRY_PASSWORD` (задать в Variables)

GitLab CI/CD — best-practice pipeline



GitLab CI architecture



GitLab Flow



🛠️ Time to make hands dirty — Lab 1

Helm deploy через GitLab CI

Helm chart: lint → package → публикация в GitLab Package Registry через `$CI_JOB_TOKEN`.

📂 **labs/gitlab_ci_lab_helm.md**

Part 4: Структура `.gitlab-ci.yml`

Stages, jobs, dependencies, variables, artifacts

Создание файла `.gitlab-ci.yml`

```yaml
# файл в корне репозитория
stages:
  - build
  - test
  - deploy

variables:
  APP_NAME: "my-app"    # глобальные переменные
  APP_VERSION: "1.0.0"

build:
  stage: build
  script:
    - ./build_script.sh
  artifacts:
    paths:
      - build/           # передаётся дальше по pipeline

test:
  stage: test
  script:
    - ./test_script.sh
  dependencies:
    - build              # получает артефакты только из build
```

- **Расположение:** корень репозитория — GitLab парсит при каждом push
- `variables` — глобальные переменные доступны во всех jobs; можно переопределять на уровне job
- `artifacts` — явная передача файлов между jobs; без них каждый job стартует «чистым»
- `dependencies` — ограничивает, чьи артефакты скачать; по умолчанию — все предыдущие
- **Совет:** начинайте с минимального (build + test), смотрите pipeline, расширяйте итерационно
- **Валидация:** GitLab UI → CI/CD → Editor (встроенный линтер) или `glab ci lint`

Стадии (stages)

```yaml
stages:
  - build
  - test
  - deploy
```

Правило: все jobs одного stage запускаются **параллельно**, stages — строго **последовательно**.

Если хоть один job в stage упал → следующий stage **не запустится**.

Default stages:

`.pre` · `build` · `test` · `deploy` · `.post`



Задачи (jobs) — сборка

```yaml
build:
  stage: build          # к какому stage относится
  script:
    - ./build_script.sh # команды выполняются последовательно
  artifacts:
    paths:
      - build/          # сохранить каталог для следующих jobs
```

Job = единица выполнения внутри stage.
Каждый job — отдельный процесс на runner'е (Docker-контейнер или K8s pod).

- `stage` — определяет порядок; jobs одного stage запускаются параллельно
- `script` — массив команд; при ненулевом exit-коде любой команды job падает
- `artifacts:paths` — единственный способ передать файлы следующему job без cache
- **Изоляция:** между jobs нет общей ФС — артефакты и cache передаются явно
- **Что класть в artifacts:** бинарники, bundle, отчёты тестов, скриншоты e2e
- **Готча:** артефакты хранятся на GitLab сервере — большие файлы замедляют pipeline

Задачи (jobs) — тестирование

```yaml
test:
  stage: test
  script:
    - ./test_script.sh   # запускаем тесты
  dependencies:
    - build              # скачиваем артефакты только из build
```

`dependencies`	Поведение
(не задан)	артефакты от **всех** предыдущих jobs
`[build]`	только из `build`
`[]`	не скачивать ничего

- `dependencies` контролирует, чьи артефакты попадут в рабочую директорию job
- **По умолчанию** job получает артефакты от всех предыдущих jobs — может быть лишним
- **Оптимизация:** `dependencies: []` на jobs, которым файлы не нужны — ускоряет старт
- **Артефакты** автоматически распаковываются в рабочую директорию перед запуском `script`
- **Реальный кейс:** e2e нужен только `frontend-build`, а не все артефакты монорепо
- **Готча:** `dependencies` ≠ `needs` — `needs` меняет порядок запуска, `dependencies` — только файлы

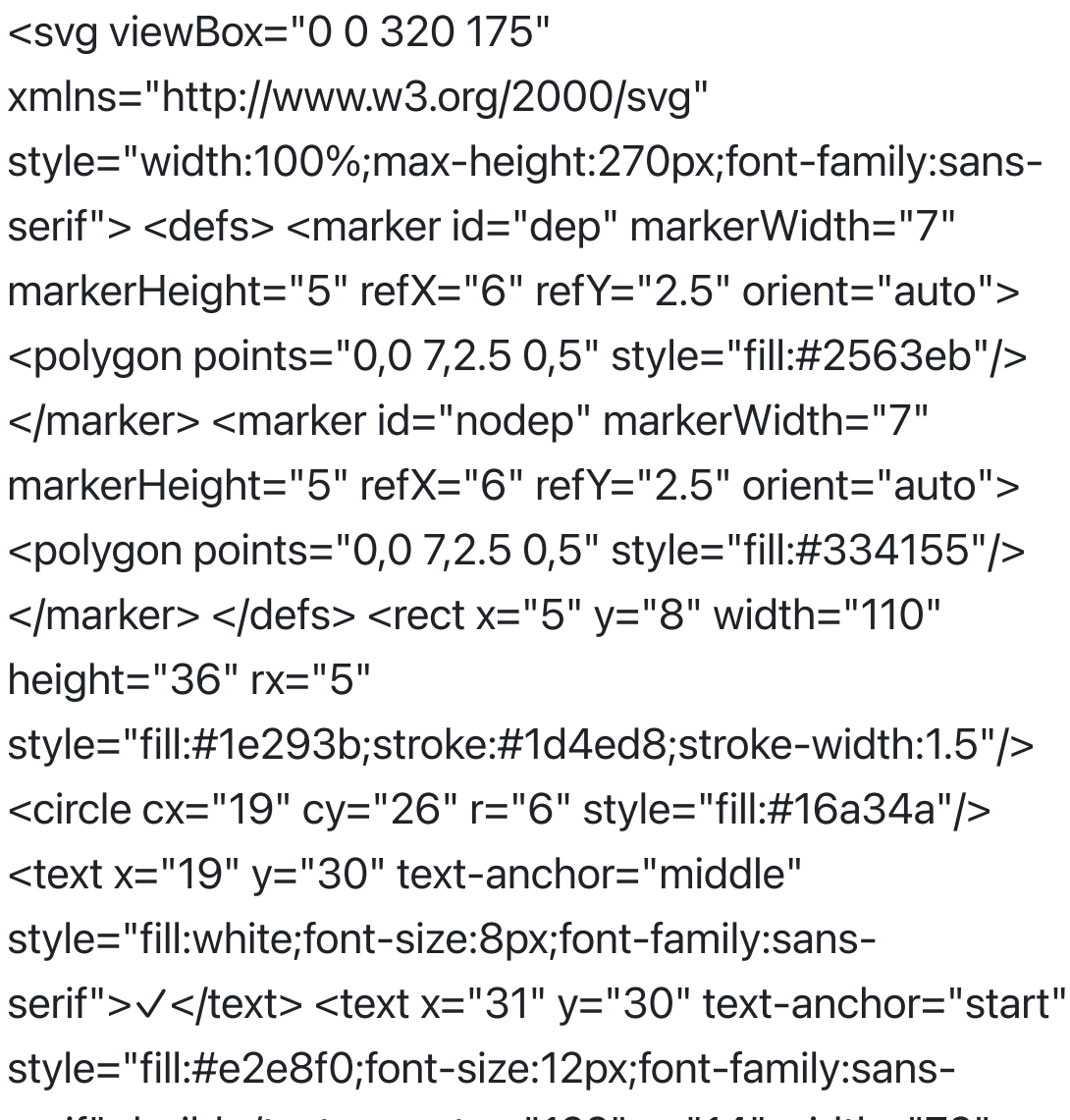
Зависимости (dependencies)

Позволяют job скачивать **артефакты** из предыдущих jobs.

```yaml
test:
  stage: test
  dependencies:
    - build        # скачать только артефакты build
  script:
    - ./run-tests

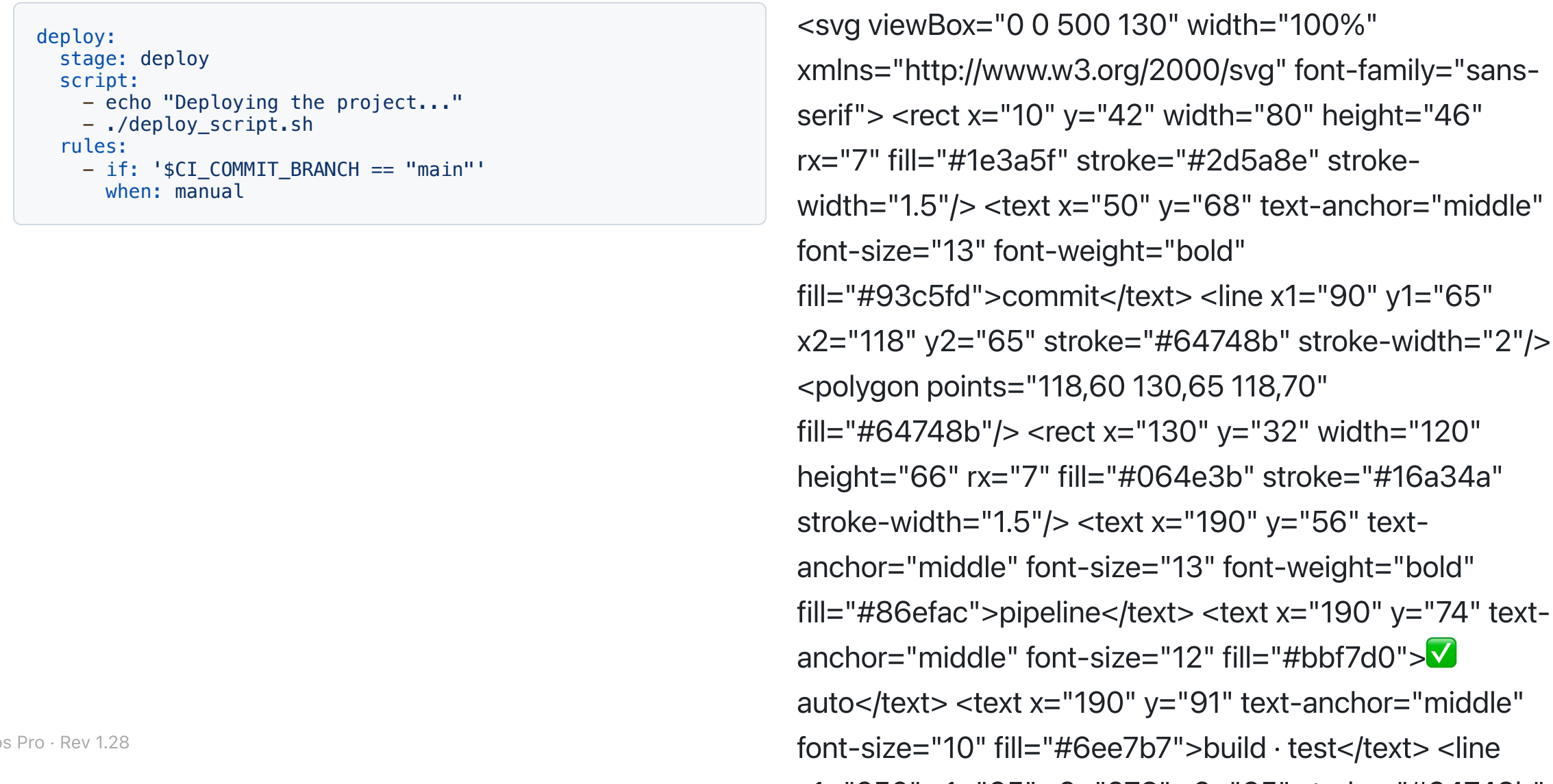
deploy:
  stage: deploy
  dependencies: [] # не скачивать ничего
  script:
    - ./deploy
```

> По умолчанию job скачивает артефакты **всех** предыдущих jobs — `dependencies: []` отключает это.



Задача развертывания (deploy)

```yaml
deploy:
  stage: deploy
  script:
    - echo "Deploying the project..."
    - ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```



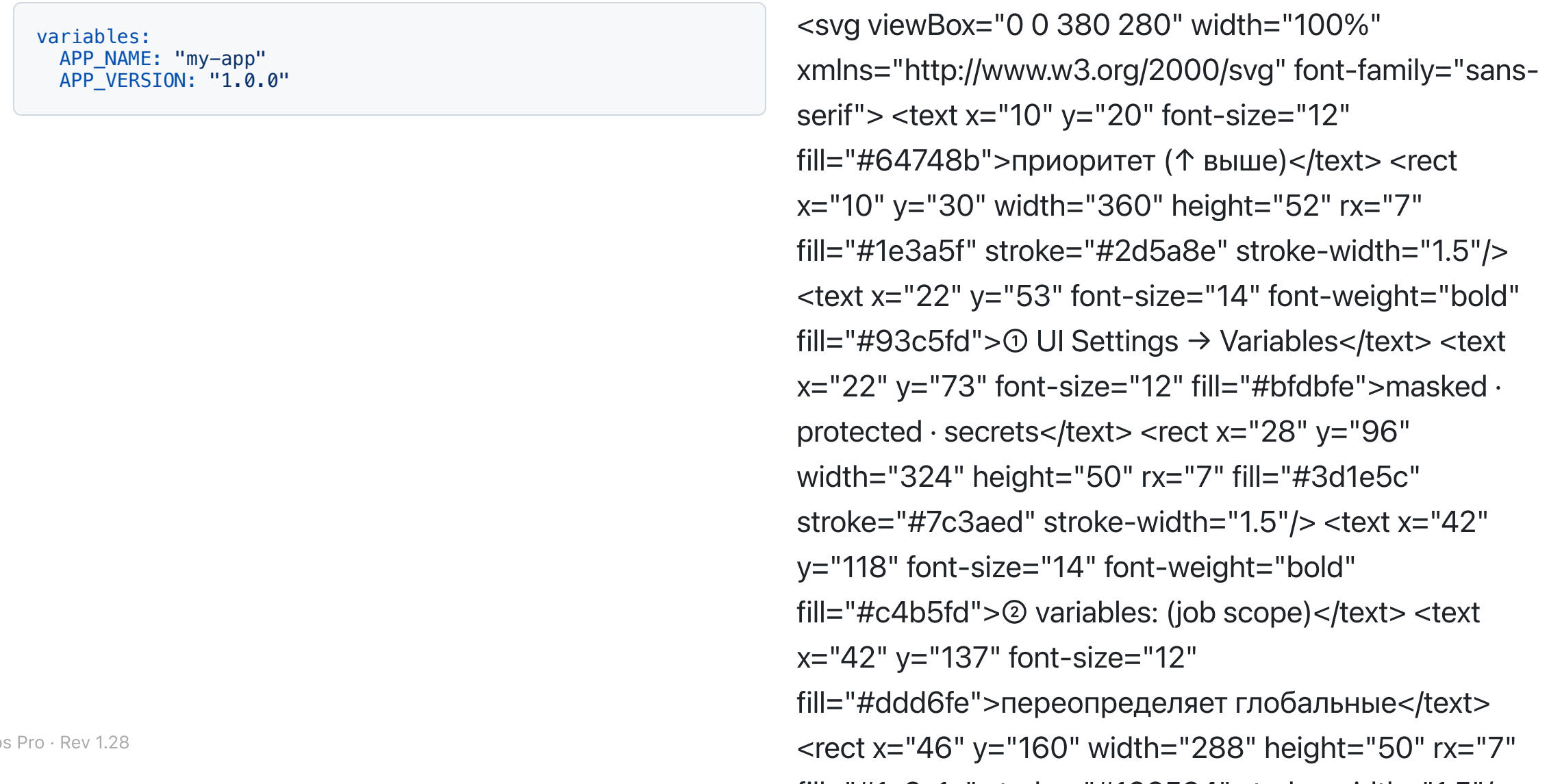
Триггеры (triggers)

```yaml
deploy:
  stage: deploy
  script:
    - echo "Deploying the project..."
    - ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/'
      when: on_success
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
    - when: never
```



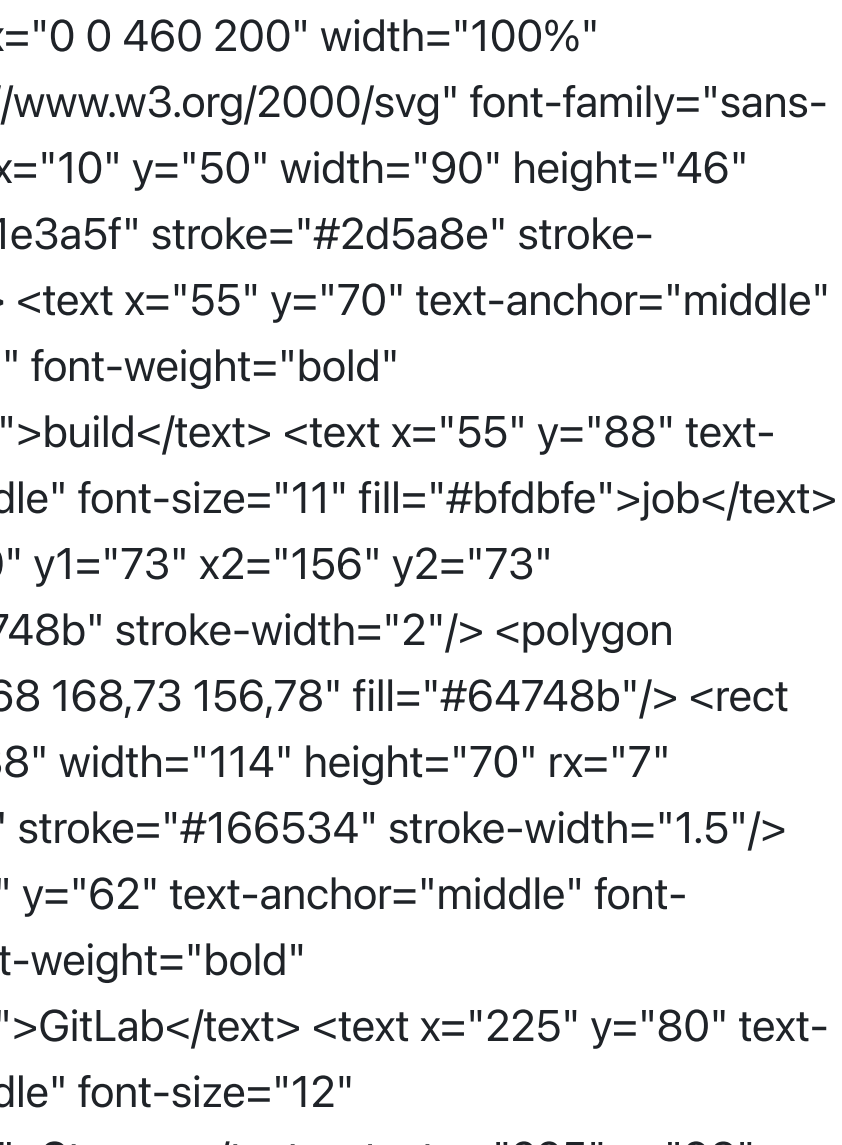
Переменные окружения

```yaml
variables:
  APP_NAME: "my-app"
  APP_VERSION: "1.0.0"
```



Артефакты

```yaml
artifacts:
  paths:
    - build/
  expire_in: 1 week
  reports:
    junit: report.xml
```



Walkthrough — собираем pipeline по шагам

```
Шаг 1: Один test job — простейший .gitlab-ci.yml
Шаг 2: Артефакт — делаем сборку скачиваемой
Шаг 3: Stages — sequential execution (test → package)
Шаг 4: Compile stage — не дублируем работу
Шаг 5: Parallel — pack-gz и pack-iso в одном stage
Шаг 6: DAG через `needs` — out of order
```

> 💡 Каждый шаг — маленькое улучшение. Так работает adoption в реальной команде.

- **Обучающая лестница:** от `script:` до полноценного DAG pipeline
- Каждый шаг — ответ на проблему предыдущего
- Конкретные YAML можно скопировать и попробовать прямо сейчас
- Финальный вывод: **stages создают барьеры, `needs` их убирает**
- Паттерн подходит для объяснения incremental CI adoption команде

Шаг 1 — первый test job

```yaml
test:
  before_script:
    - echo "Hello " | tr -d "\n" > file1.txt
    - echo "world" > file2.txt
  script:
    # grep -q: тихий режим, exit 0 = найдено, 1 = нет
    - cat file1.txt file2.txt | grep -q 'Hello world'
```

Минимальный валидный `.gitlab-ci.yml` — один job, одна команда.

- `script:` — **обязательное** поле каждого job'а
- `before_script:` — setup перед основными командами; каждая строка — отдельная shell-команда
- `grep -q` работает как assertion: exit 0 / 1 → job ✅ / ❌
- Default image `ruby:3.1` (~900 MB) — на реальных проектах сразу берут lightweight
- **Gotcha:** поменяйте `world` → `World` — grep вернёт 1, job упадёт. Первый automated test!

Шаг 2 — артефакты сборки

```yaml
test:
  script: cat file1.txt file2.txt | grep -q 'Hello world'

package:
  script: cat file1.txt file2.txt | gzip > packaged.gz
  artifacts:
    paths:
      - packaged.gz        # что сохранить
    # expire_in: 1 week   # TTL (по умолчанию — 30 дней, настраивается в Admin Area)
    # when: always        # сохранять даже при падении job
```

> ⚠️ Без `artifacts:` файл `packaged.gz` будет **потерян** — workspace удаляется после job

- Каждый job — **эфемерное окружение**: контейнер уничтожается после завершения
- `artifacts.paths` → GitLab сохраняет файлы на storage; доступны через UI и API
- `paths:` поддерживают glob: `build/*.jar`, `**/*.log`
- `when: always` — сохранять даже при упавшем job (для debug-логов)
- **Gotcha:** `paths: ["."]` сохранит всё, но создаст огромный artifact — не злоупотребляйте

Шаг 3 — sequential execution через `stages`

```yaml
stages:
  - test      # барьер 1
  - package   # барьер 2

test:
  stage: test
  script: cat file1.txt file2.txt | grep -q 'Hello world'

package:
  stage: package
  script: cat file1.txt file2.txt | gzip > packaged.gz
  artifacts:
    paths:
      - packaged.gz
```

Правило: все jobs в `package` ждут окончания **всех** jobs в `test`.

- `stages:` — список барьеров синхронизации; jobs внутри stage стартуют **параллельно**
- Если хоть один job в stage ❌ → следующий stage **пропускается**
- Гарантия порядка: не задеплоим то, что не прошло тесты
- Default stages (без явного `stages:`): `.pre → build → test → deploy → .post`
- **Gotcha:** если один job из 5 идёт 10 мин — весь следующий stage ждёт 10 мин → motivation для `needs`

Шаг 4 — общий артефакт между jobs

```yaml
stages: [compile, test, package]

compile:
  stage: compile
  script: cat file1.txt file2.txt > compiled.txt
  artifacts:
    paths: [compiled.txt]
    expire_in: 20 minutes   # временный артефакт

test:
  stage: test
  script: cat compiled.txt | grep -q 'Hello world'
  # compiled.txt загружается автоматически

package:
  stage: package
  script: cat compiled.txt | gzip > packaged.gz
  artifacts:
    paths: [packaged.gz]
  # dependencies: [compile]  # явно ограничить источник
```

- Паттерн **"compile once, use many"**: собираем один раз — используют все downstream jobs
- Jobs автоматически скачивают artifacts всех **предыдущих** stages
- `dependencies: [compile]` — скачивать только конкретный job; `dependencies: []` — ничего не скачивать
- **Короткий** `expire_in` (20 мин) — для промежуточных artifacts; долгий — для release/audit
- **Gotcha:** Java `mvn package` × 3 jobs vs × 1 job + artifact = экономия 2/3 build-времени

Шаг 4 — общий артефакт между jobs

```yaml
stages:
  - compile
  - test
  - package

compile:
  stage: compile
  script: cat file1.txt file2.txt > compiled.txt
  artifacts:
    paths:
      - compiled.txt
    expire_in: 20 minutes  # удалится авто

test:
  stage: test
  script: cat compiled.txt | grep -q 'Hello world'
  # compiled.txt пришёл автоматически

package:
  stage: package
  script: cat compiled.txt | gzip > packaged.gz
  artifacts:
    paths:
      - packaged.gz
```

Паттерн «compile once, use many»

- Jobs наследуют artifacts всех предыдущих stages — **по умолчанию**, явно указывать не нужно
- `dependencies: [compile]` — взять только от конкретного job; `dependencies: []` — ничего не скачивать
- `expire_in: 20 minutes` — для промежуточных артефактов (bytecode, transpiled JS); релизные artifacts держать дольше или без TTL
- Реальный пример: `mvn package` создаёт jar один раз → test и deploy берут тот же файл, не пересобирают
- **Экономия**: один тяжёлый build вместо N повторных запусков

Шаг 5 — parallel jobs в одном stage

```yaml
# Оба job'а в stage: package → стартуют одновременно
pack-gz:
  stage: package
  script: cat compiled.txt | gzip > packaged.gz
  artifacts:
    paths: [packaged.gz]

pack-iso:
  stage: package
  before_script:
    - apk add xorriso   # установка инструмента
  script:
    - mkisofs -o ./packaged.iso ./compiled.txt
  artifacts:
    paths: [packaged.iso]
```

> Wall-clock = **max**(pack-gz, pack-iso), не sum

Параллельность через несколько jobs в одном stage

- Все jobs одного stage запускаются **одновременно** — GitLab не ждёт завершения одного перед стартом другого
- Оба job'а получают `compiled.txt` из artifacts предыдущего stage
- Применяется для: кросс-платформенной сборки (amd64/arm64), multi-format packaging (tar/deb/rpm/docker), параллельных test-шардов
- **Ограничение**: реальный параллелизм ограничен размером runner-pool (`concurrent` в config.toml); маленький пул → очередь вместо параллельности
- Масштабирование: autoscaling GitLab Runner на k8s решает проблему pool-лимитов

Полный pipeline — собираем всё вместе

```yaml
image: alpine          # ① default image

stages: [compile, test, package]

compile:
  stage: compile
  script: # ... собрать файлы
  artifacts:
    paths: [compiled.txt]
    expire_in: 20 minutes  # ③

test:
  stage: test
  script: # ... проверить compiled.txt

pack-gz:              # ④ параллельно
  stage: package
  script: # ... gzip
  needs: ["test"]    # ⑤ DAG
  artifacts:
    paths: [packaged.gz]

pack-iso:             # ④ параллельно
  stage: package
  script: # ... mkisofs
  needs: ["test"]    # ⑤ DAG
  artifacts:
    paths: [packaged.iso]
```

Что здесь показано:

① `image:` на уровне pipeline — default для всех jobs

② 3 stages последовательно: `compile → test → package`

③ `artifacts` с TTL 20 мин — передаёт `compiled.txt` в `test`

④ `pack-gz` и `pack-iso` — параллельно в одном stage

⑤ `needs: ["test"]` — DAG, стартуют как только `test` ✅

Timeline:

```
t=0   compile ▶
t=10s test    ▶
t=20s pack-gz ▶ pack-iso ▶  (одновременно)
t=50s pipeline ✅
```

🛠️ Time to make hands dirty — Walkthrough

Собираем pipeline по шагам

От одного job до DAG с `needs` — шесть шагов, шесть коммитов, шесть pipeline-ов.

📂 **labs/gitlab_ci_lab_walkthrough.md**

Part 5: Rules — условия запуска

Гибкое управление выполнением jobs

Part 5: Rules — условия запуска

Гибкое управление выполнением jobs

Pipeline Examples with rules

```yaml
build:
  stage: build
  script: ./build_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_BRANCH == "develop"'

test:
  stage: test
  script: ./test_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
    - if: '$CI_COMMIT_BRANCH == "develop"'

deploy:
  stage: deploy
  script: ./deploy_script.sh
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'  # только main!
```

`rules` — современная замена `only` / `except`

- Условия проверяются **сверху вниз**, первое подходящее срабатывает; ни одно не подошло → job не запускается
- В примере: `build` и `test` идут на `main` и `develop`; `deploy` — **только на `main`**
- Комбинируется с `when:` для тонкой настройки: `on_success`, `manual`, `never`
- Регулярки в `if:`: `$CI_COMMIT_BRANCH =~ /^release\//` — ветки по паттерну
- **Gotcha**: порядок правил важен — первое совпадение выигрывает, остальные игнорируются

Example with `rules:if` + pipeline source

```yaml
build:
  stage: build
  script: [./build_script.sh]
  rules:
    - if: '$CI_PIPELINE_SOURCE == "push"'          # обычный push
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"' # MR pipeline

test:
  stage: test
  script: [./test_script.sh]
  rules:
    - if: '$CI_PIPELINE_SOURCE == "push"'
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'

deploy:
  stage: deploy
  script: [./deploy_script.sh]
  rules:
    - if: '$CI_PIPELINE_SOURCE == "schedule"'      # только по расписанию
```

`$CI_PIPELINE_SOURCE` — кто запустил pipeline

- `push` — обычный push в ветку
- `merge_request_event` — pipeline на MR
- `schedule` — cron-триггер
- Также: `trigger`, `web`, `parent_pipeline`, `pipeline`

Паттерн из примера:

- build/test — на push **и** MR → быстрый feedback в MR
- deploy — **только** по расписанию (staging ночью)

Типичные use-cases для `schedule`:

- nightly e2e tests
- security scans (Trivy)
- dependency updates

Example with `rules:exists`

```yaml
build:
  stage: build
  script: [./build_script.sh]
  rules:
    - exists:
      - Dockerfile          # job запустится только если файл есть

test:
  stage: test
  script: [./test_script.sh]
  rules:
    - exists:
      - tests/test_script.sh

deploy:
  stage: deploy
  script: [./deploy_script.sh]
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'  # if + exists в одном элементе = AND
      exists:
        - deploy/deploy_script.sh
```

`exists` — job только если файл присутствует в репозитории

- Делает pipeline **самодокументируемым**: есть Dockerfile → запускаем build
- Без `exists` job просто упал бы; с ним — элегантно пропускается

Типичные сценарии:

- Reusable template через `include:` — каждый проект активирует нужные jobs своими файлами
- Монорепо: разные сервисы включают разные jobs
- Постепенное внедрение: job появляется, когда команда добавит нужный файл

Gotcha: `if` и `exists` внутри одного `rules`-элемента работают как AND

🛠️ Time to make hands dirty — Rules

Управление запуском jobs через rules

Практикуем все механизмы `rules` : if, changes, exists, pipeline source, workflow.

📂 **labs/gitlab_ci_lab_rules.md**

Part 6: Расширенные возможности

YAML anchors, extends, triggers, parallel

YAML anchors

```yaml
# &name — объявление якоря (hidden job — не создаёт pipeline job)
.job_template: &job_configuration
  image: ruby:2.6
  services:
    - postgres
    - redis

# <<: *name — merge keys из якоря
test1:
  <<: *job_configuration  # подмешиваем image + services
  script:
    - test1 project

test2:
  <<: *job_configuration
  script:
    - test2 project
```

YAML anchors — встроенный DRY для повторяющихся блоков

- `&name` — объявить якорь, `*name` — сослаться, `<<:` — merge map
- Hidden jobs (`.prefix`) не создают реальный job — только шаблон
- Устраняет дублирование `image`, `services`, `tags`, `before_script`

Ограничение: работает только **в пределах одного файла**

- Переиспользование между файлами (`include:`) — не работает
- Для cross-file DRY используется `extends`

Правило: если anchors стали длинными или нужны в нескольких файлах — переходи на `extends`

`extends` — переиспользование конфигурации

```yaml
# Базовый шаблон: только rules
.tests:
  rules:
    - if: $CI_PIPELINE_SOURCE == "push"

# Наследует .tests, добавляет script
.rspec:
  extends: .tests
  script: rake rspec

# Конкретные jobs: переопределяют variables
rspec 1:
  variables:
    RSPEC_SUITE: '1'
  extends: .rspec  # ← .rspec ← .tests

rspec 2:
  variables:
    RSPEC_SUITE: '2'
  extends: .rspec

spinach:
  extends: .tests  # напрямую от .tests
  script: rake spinach
```

`extends` — GitLab-native DRY, умнее YAML anchors

- Поддерживает **цепочку наследования**: `rspec 1` ← `.rspec` ← `.tests`
- **Глубокий merge** (anchors делают shallow)
- **Множественное наследование**: `extends: [.base, .with_db]`
- **Работает между файлами** через `include:` — главное преимущество

Паттерн:

- `.шаблон` (hidden) — общая конфигурация
- Реальные jobs наследуют и точечно переопределяют

Итог: в современных проектах `extends` заменяет anchors в 95% случаев

`trigger:include` — child pipelines

```yaml
trigger-child-pipeline:
  trigger:
    include: path/to/child-pipeline.gitlab-ci.yml
    # strategy: depend  # раскомментить для синхронизации
```

Варианты `include:`:

- Статичный файл в том же репо
- Сгенерированный артефакт (`include:artifact`)
- Внешний URL или шаблон компонента

Что даёт `trigger:include`

- Запускает **child pipeline** в том же проекте на том же коммите
- Родитель немедленно помечает job как `success` — child работает асинхронно
- Child имеет собственные `stages`, независимые от родителя
- Переменные родителя попадают в child по умолчанию

Когда использовать:

- Монорепо: каждый сервис — свой child pipeline
- Dynamic pipelines: генерируем `.gitlab-ci.yml` на лету → `trigger:include:artifact`
- Логическое разделение: security scans отдельно от сборки

> Нужна синхронизация? Добавьте `strategy: depend`

`trigger:project` — multi-project pipelines

```yaml
# Запуск на default branch
trigger-default:
  trigger:
    project: my-group/my-project  # другой репозиторий

# Запуск на конкретном branch
trigger-dev:
  trigger:
    project: my-group/my-project
    branch: development
    strategy: depend      # ждём результат
  variables:
    TRIGGERED_BY: $CI_PROJECT_NAME  # прокидываем контекст
```

Что даёт `trigger:project`

- Запускает pipeline **в другом репозитории** — полная независимость кодовых баз
- По умолчанию стартует на default branch целевого проекта; другую ветку задаёт `branch:`
- Авторизация через `CI_JOB_TOKEN` — целевой проект должен разрешить trigger в **Settings → CI/CD → Token Access**

Типичные сценарии:

- Микросервисы: gateway обновился → триггерит интеграционные тесты
- E2E pipeline в отдельном проекте — вызывается из всех app-репо
- Инфраструктура: после деплоя app → обновление конфигов в infra-репо

> Передавайте `variables:` чтобы child знал, кто его запустил и с каким контекстом

`trigger:strategy: depend`

```yaml
trigger_job:
  trigger:
    include: path/to/child-pipeline.yml
    strategy: depend
```

Когда применять:

- Нужна синхронизация перед `deploy`
- Fail-fast: child упал — нет смысла продолжать
- Parent pipeline должен показать итог child

> ⚠ Передаётся только **статус** — файлы и переменные child обратно не возвращаются



`trigger:forward` — передача переменных

```yaml
variables:
  VAR: value          # yaml_variable

# Default: VAR передаётся, MYVAR — нет
child1:
  trigger:
    include: .child-pipeline.yml

# Пробросить pipeline variables (ручной запуск / API)
child2:
  trigger:
    include: .child-pipeline.yml
    forward:
      pipeline_variables: true  # передать MYVAR

# Изолировать child от YAML variables
child3:
  trigger:
    include: .child-pipeline.yml
    forward:
      yaml_variables: false     # VAR не передаётся
```

Два типа переменных и их defaults

Тип	Источник	Default
`yaml_variables`	`variables:` в `.gitlab-ci.yml`	**true**
`pipeline_variables`	ручной запуск / API	**false**

Зачем менять defaults:

- `pipeline_variables: true` — передать `DEPLOY_ENV=prod` из ручного запуска в child
- `yaml_variables: false` — изоляция: child использует собственные credentials, не наследует родительские

Gotcha: forwarded переменные имеют **высокий приоритет** в child — перекрывают одноимённые переменные child-файла

> Секреты не пробрасывайте — задавайте в child через protected variables напрямую

Sharing data parent ↔ child

Problem: `trigger:forward` пробрасывает только переменные, известные ДО старта parent. А что насчёт значений, **вычисленных в runtime** — image tag, build ID, артефакт?

Три способа поделиться:

- `dotenv` + `needs:project` — runtime variables
- Package Registry — артефакты любого размера
- `inputs` — типизированные параметры

Когда что выбрать:

- знаешь значение заранее → `inputs`
- считаешь в job → `dotenv`
- большой файл/бинарник → Package Registry

Pattern A — `dotenv` artifacts

```yaml
stages: [build, triggers]

create_var_job:
  stage: build
  script:
    # Вычисляем значение в runtime
    - echo "var_1=Hi, I'm a Parent var" >> var.env
  artifacts:
    reports:
      dotenv: var.env   # ключ: GitLab парсит KEY=value

telco_service_a:
  stage: triggers
  trigger:
    include: service_a/.gitlab-ci.yml
  rules:
    - changes: [service_a/*]  # монорепо: только при изменениях
```

Как работает `dotenv`:

- `artifacts:reports:dotenv` — не обычный файл, а контракт: GitLab парсит `KEY=value` и превращает в CI/CD переменные
- downstream-jobs получают `$var_1` автоматически через `needs`
- Формат строгий: `KEY=value`, без пробелов и кавычек по краям

Типичные use-cases:

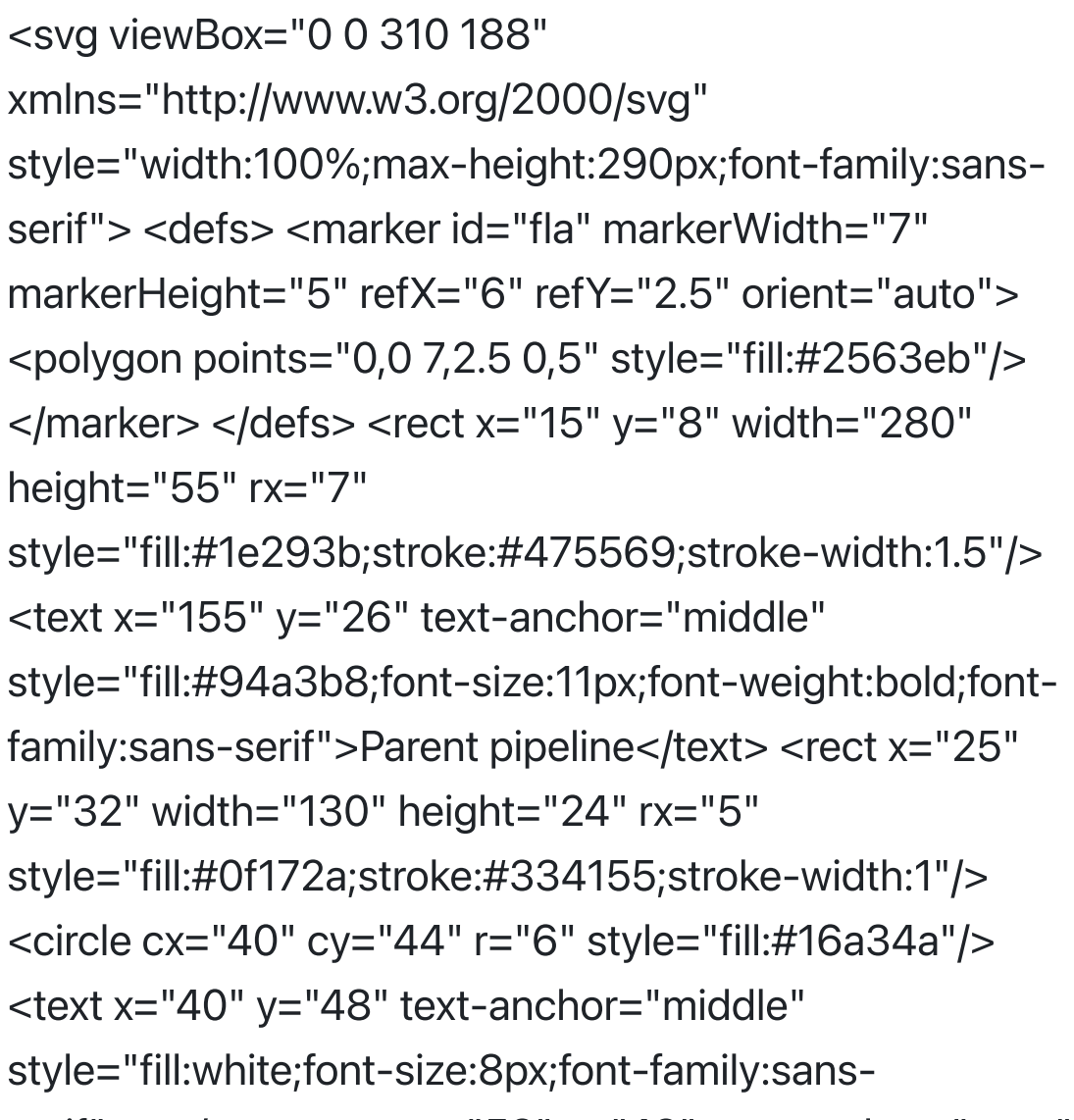
- `IMAGE_TAG=registry/app:abc123` → child деплоит нужный образ
- `NEW_VERSION=1.4.2` → release jobs знают версию

Gotcha: dotenv-переменные перезаписывают одноимённые YAML `variables` — обычно желаемо, но может удивить

Pattern A — child consumes via `needs`

```yaml
# service_a/.gitlab-ci.yml
build_a:
  stage: build
  script:
    - echo $var_1
  needs:
    - project: gitlab-da/use-cases/parent-pipeline
      job: create_var_job
      ref: main
      artifacts: true
```

> ⚠️ `needs:project` для cross-pipeline dotenv — **только Premium/Ultimate**



Real-world — Docker image tag

```yaml
# App-repo (parent)
build-prod-image:
  stage: build
  script:
    - docker build -t $PRODUCTION_IMAGE .
    - docker push $PRODUCTION_IMAGE
    # Экспортируем тег в dotenv — он известен только ПОСЛЕ build
    - echo "UPSTREAM_CONTAINER_IMAGE=$PRODUCTION_IMAGE" >> prodimage.env
  artifacts:
    reports:
      dotenv: prodimage.env

# Manifests-repo (child, отдельный проект)
update-helm-values:
  script:
    - yq eval -i '.spec.template.spec.containers[0].image = env(UPSTREAM_CONTAINER_IMAGE)' deploy.yaml
    - git commit -am '[skip ci] prod image update'  # [skip ci] — критично!
    - git push origin main
  needs:
    - project: org/app-repo   # cross-project
      job: build-prod-image
      ref: main
      artifacts: true
```

Канонический GitOps-flow:

1. App-repo строит образ → пишет тег в `prodimage.env`
2. Manifests-repo подтягивает тег через `needs:project`
3. `yq` патчит `image:` в Helm values
4. Коммит в main → ArgoCD/Flux деплоит

Ключевые детали:

- `[skip ci]` в commit-сообщении — **обязателен**: без него manifests-repo триггерит сам себя → бесконечный цикл
- `needs:project` — только **Premium/Ultimate**; на CE используй Pattern B (Package Registry)
- Сломал имя job в app-repo → сломал deployment в child — это tight coupling

Когда применять: GitOps с отдельным manifests-repo + нужна аудитность деплоев через git history

Pattern B — Package Registry (CE-friendly)

```yaml
# Parent pipeline
variables:
  PACKAGE_NAME: "pipeline-artifacts"
  PACKAGE_VERSION: "$CI_PIPELINE_ID"  # уникальна per-run
  ARTIFACT_FILE: "artifact.txt"

create-and-upload-artifact:
  stage: create-upload
  script:
    - bash scripts/create-artifact.sh
    - bash scripts/upload-to-registry.sh  # curl → Generic Packages API

trigger-child:
  stage: trigger
  trigger:
    include: child-pipeline.yml
    strategy: depend     # parent ждёт child → общий статус осмысленный
  variables:
    PARENT_PIPELINE_ID: $CI_PIPELINE_ID  # «координата» для скачивания
```

> Works on **Community Edition** — только HTTP + `CI_JOB_TOKEN`

Идея: Package Registry — HTTP-хранилище внутри GitLab. Передаём child «координату» (`PIPELINE_ID`) — он знает, откуда скачать.

vs. dotenv:

	dotenv	Package Registry
Размер	~5 KB	без лимита
Тип данных	key=value	любой файл
Tier	CE	CE
Авто-инъекция	да	нет

Gotchas:

- Переменная не «прорастает» автоматически — нужно парсить файл вручную
- Registry не чистится сам → нужна **retention policy** (Settings → Packages)
- Для cross-project upload — разрешить JOB-TOKEN в Token Access settings целевого проекта

Pattern B — upload & download scripts

```
# upload-to-registry.sh (в parent)
UPLOAD_URL="${CI_API_V4_URL}/projects/${CI_PROJECT_ID}\
/packages/generic/${PACKAGE_NAME}/${PACKAGE_VERSION}/${ARTIFACT_FILE}"

curl --fail-with-body \
  --header "JOB-TOKEN: $CI_JOB_TOKEN" \
  --upload-file "$ARTIFACT_FILE" \
  "$UPLOAD_URL"
```

```
# download-from-registry.sh (в child)
DOWNLOAD_URL="${CI_API_V4_URL}/projects/${CI_PROJECT_ID}\
/packages/generic/${PACKAGE_NAME}/${PACKAGE_VERSION}/${ARTIFACT_FILE}"

curl --fail-with-body \
  --header "JOB-TOKEN: $CI_JOB_TOKEN" \
  --output "$ARTIFACT_FILE" \
  "$DOWNLOAD_URL"
```

Три критичных детали:

- `--fail-with-body` — curl завершается с ненулевым кодом при HTTP 4xx/5xx; без этого pipeline зелёный при сломанной загрузке
- `JOB-TOKEN: $CI_JOB_TOKEN` — токен привязан к job, валиден ~1 час, авторизация бесплатная
- `PACKAGE_VERSION=$CI_PIPELINE_ID` — уникальная версия per-run, не затирает предыдущие

Структура URL:

```
/api/v4/projects/<id>/packages/generic
  /<name>/<version>/<filename>
```

Retention: без cleanup policy registry разрастается. Настрой: Settings → Packages → cleanup rules

Один `curl` = один файл. Для нескольких артефактов — разные имена файлов в той же версии.

Pattern C — `inputs` (typed parameters)

Современный способ — типизированные параметры:

```yaml
# Parent
trigger-staging:
  trigger:
    include:
      - local: service_a/.gitlab-ci.yml
        inputs:
          environment: staging
          version: "1.0.0"
```

```yaml
# Child (service_a/.gitlab-ci.yml)
spec:
  inputs:
    environment:
      description: "Deployment environment"
      options: [staging, production]
    version:
      type: string
      description: "Application version"
---
deploy:
  script:
    - echo "Deploying $[[ inputs.version ]] to $[[ inputs.environment ]]"
```

Variables — скрытые издержки

Variables — это всегда строки. GitLab не валидирует их до runtime — и тогда уже поздно.

Что ломается:

- нет type validation
- runtime mutability
- размытые границы с secrets
- ошибки находим через 30 минут pipeline'а

Боль на практике:

- `MAX_RETRIES="3"` → `$((MAX_RETRIES+1))` = `"31"`
- `ENABLE_TESTS="true"` → `[ "$X" = true ]` всегда false
- одна опечатка в имени переменной → silent skip

Precedence trap — почему variables рушат шаблоны

```yaml
# .gitlab-ci.yml
variables:
  ENVIRONMENT: production
  DATABASE_URL: prod-db.example.com  # глобальная

include:
  - local: 'templates/test-template.yml'   # переопределяет ENVIRONMENT: test
  - local: 'templates/deploy-template.yml' # молча наследует DATABASE_URL
```

Результат: тесты бьют по `prod-db.example.com`.
Пайплайн — зелёный.

Что происходит

- Порядок приоритета: `job-level` > `global` > `include` — шаблон переопределяет `ENVIRONMENT`, но не `DATABASE_URL`
- Шаблон **не знает**, какие переменные придут сверху — нет явного контракта
- Ошибка обнаруживается через логи или инцидент, не при парсинге

Реальные последствия

- Миграции на prod из test-pipeline
- `terraform destroy` не на той среде

Решение — `inputs:`

- Каждый template **декларирует** параметры
- Несовпадение → ошибка **до** запуска jobs

Inputs — один template, много окружений

```yaml
# templates/environment-template.yml
spec:
  inputs:
    environment:
      type: string
    database_url:
      type: string
    action:
      type: string
---
$[[ inputs.action ]]-$[[ inputs.environment ]]:
  script:
    - echo "Running $[[ inputs.action ]] in $[[ inputs.environment ]]"
    - run-$[[ inputs.action ]] --env=$[[ inputs.environment ]] --db=$[[ inputs.database_url ]]
```

→ Имя job'а тоже параметризуется: `tests-test`, `deploy-production` из одного файла.

Type safety — boolean & number в деле

```yaml
# variables: — скрытая поломка
variables:
  ENABLE_TESTS: "false"  # строка
  MAX_RETRIES: "3a"      # опечатка, никто не проверит
script:
  - if [ -n "$ENABLE_TESTS" ]; then ...  # "false" непустая → тесты всё равно идут
  - retry=$((MAX_RETRIES + 1))           # ошибка только в runtime
```

```yaml
# inputs: — корректно
spec:
  inputs:
    enable_tests:
      type: boolean   # GitLab валидирует при парсинге
      default: true
    max_retries:
      type: number
      default: 3
script:
  - if [ "$[[ inputs.enable_tests ]]" = true ]; then ...  # OK
  - retry=$(($[[ inputs.max_retries ]] + 1))              # 4
```

Bug #1 — boolean comparison

- В `variables:` всё — строки: `"false"`, `"no"`, `"0"` легко принять за «включено» в проверке вида `[ -n "$X" ]`
- GitLab не валидирует значение: `"ture"` пройдёт молча
- `type: boolean` — GitLab принимает только `true` / `false` и подставляет буквальное значение, сравнение `= true` надёжное

Bug #2 — arithmetic со строкой

- `MAX_RETRIES: "3a"` (опечатка) → runtime error в bash
- `type: number` → GitLab **отказывается парсить** pipeline если значение не число

Доступные типы inputs

- `string`, `number`, `boolean`, `array`
- `options:` — whitelist значений
- `regex:` — валидация форматом

`expand_vars` — variables in rules

Проблема: `inputs` резолвятся при парсинге, но иногда нужно подставить runtime variable в правило.

```yaml
spec:
  inputs:
    target_environment:
      description: "Target deployment environment"
      default: "${CI_COMMIT_REF_SLUG}"
---
deploy-job:
  rules:
    - if: '"$[[ inputs.target_environment | expand_vars ]]" == "production"'
      variables:
        DEPLOY_MODE: "blue-green"
        APPROVAL_REQUIRED: "true"
    - when: always
      variables:
        DEPLOY_MODE: "rolling"
  script:
    - echo "Target: $[[ inputs.target_environment | expand_vars ]]"
```

Function chaining — `expand_vars | truncate`

Фильтры можно цеплять:

```
spec:
  inputs:
    service_identifier:
      default: 'service-$CI_PROJECT_NAME-$CI_COMMIT_REF_SLUG'
---
create-resource:
  script:
    - resource_name=$[[ inputs.service_identifier | expand_vars | truncate(0,50) ]]
```

Результат: `service-my-very-long-project-feature-branch-name` → обрезан до 50 символов.

> 💡 Полезно для AWS/GCP ресурсов, у которых жёсткий лимит на длину имени

Где работают inputs (17.11+)

Полный охват pipeline-сценариев:

До 17.11:

- `include:` с `inputs:`
- CI/CD Components

С 17.11+:

- **Scheduled pipelines** — UI форма с defaults и override
- **Downstream pipelines** — `trigger: include: inputs:`
- **Manual pipelines** — валидируемая форма «Run pipeline»

Adoption strategy: начинаем с новых компонентов → мигрируем критичные workflows → к Catalog.

Decision matrix — что выбрать

Сценарий	Pattern	Tier
Знаешь значение **до** старта parent (env, version)	`inputs` или `variables:`	Free
Значение **вычисляется** в parent (image tag, build ID)	`dotenv` + `needs:project`	**Premium+**
Cross-project sharing на **CE**	Package Registry + curl	Free
Большой файл / бинарник / отчёт	Package Registry	Free
Reusable child для разных окружений	`inputs` + `options:`	Free
Нужна валидация значений	`inputs` + `type:` / `options:`	Free
Секреты	masked CI/CD variables (никогда не inputs)	Free

Parallel Matrix Jobs

```yaml
test:
  stage: test
  script:
    - nvm install $NODE_VERSION && nvm use $NODE_VERSION
    - npm install && npm test
  parallel:
    matrix:
      - NODE_VERSION: ["12", "14", "16"]  # 3 параллельных запуска
```

Многомерная матрица:

```yaml
  parallel:
    matrix:
      - NODE_VERSION: ["18", "20"]
        OS: ["alpine", "debian"]  # 2×2 = 4 jobs
```

UI показывает: `test: [18, alpine]`, `test: [20, debian]`...

Что это даёт

- Один job-definition → N параллельных запусков с разными переменными
- GitLab автоматически именует jobs: `test: [12]`, `test: [14]`, `test: [16]`

Применения

- Кросс-версионное тестирование (Python/Node/Java)
- Кросс-платформенный билд (linux/amd64, linux/arm64)
- Региональный деплой (us-east-1, eu-west-1, ap-south-1)
- Шардинг тестов по N частям

Ограничения

- Максимум **200 комбинаций** на job
- Требует несколько runners или runner с `concurrent >1`

Part 7: Includes, Cache и Artifacts

Переиспользование конфига и оптимизация скорости

Part 7: Includes, Cache и Artifacts

Переиспользование конфига и оптимизация скорости

include:local

```yaml
# Полный путь от корня репозитория
include:
  - local: '/templates/.gitlab-ci-template.yml'

# Сокращённый синтаксис (один файл)
include: '.gitlab-ci-production.yml'

# Wildcard — все YAML в папке
include:
  - local: 'templates/*.yml'
```

Что это даёт

- Разбить большой `.gitlab-ci.yml` на тематические файлы (`build.yml`, `test.yml`, `deploy.yml`)
- Переиспользовать общие блоки конфига внутри одного репо
- Wildcard `*.yml` — подключить всю директорию шаблонов сразу

Важно знать

- Файл берётся из **той же ветки**, где запущен pipeline
- На MR подтянется версия из source branch
- Symlinks не работают — используйте `include:local`

`include:project` — из другого проекта

```yaml
include:
  # Один файл из другого проекта
  - project: 'my-group/my-project'
    file: '/templates/.gitlab-ci-template.yml'

  # Несколько файлов из одного проекта
  - project: 'my-group/my-subgroup/my-project-2'
    file:
      - '/templates/.builds.yml'
      - '/templates/.tests.yml'
```

Зачем это нужно

- Platform Team держит `gitlab-ci-templates` — единое место для стандартов
- Application Teams подключают шаблоны через `include`
- Обновление шаблона в одном репо = автоматическое обновление во всех проектах

Ключевые детали

- Запускающий pipeline пользователь должен иметь **доступ** к target-проекту
- Included файл может сам содержать `include:` — поддерживается chaining
- Без `ref:` берётся HEAD — **рискованно**: pinать версию через `ref:` (следующий слайд)

`include:remote`

```
include:
  - remote: 'https://gitlab.com/example-project/-/raw/main/.gitlab-ci.yml'
```

Откуда можно подключать:

- Публичный GitLab / GitHub raw URL
- Внутренний веб-сервер с шаблонами
- S3 bucket с публичным доступом

Ограничения:

- URL должен быть **публично доступен** (auth не поддерживается)
- Только **HTTPS**
- Расширение `.yml` или `.yaml`

Когда использовать `include:project` вместо `include:remote`

Для GitLab-проектов всегда предпочтительнее `include:project`:

	`remote`	`project`
Auth	❌ нет	✅ токен
Ref pinning	❌ нет	✅ `ref:`
Private repo	❌ нет	✅ да

⚠️ **Security:** `include:remote` = выполнение чужого кода в вашем CI.
Аналог `curl | bash` — только после тщательного review.

`default` configuration from included file

`/templates/.before-script-template.yml`:

```yaml
default:
  before_script:
    - apt-get update -qq && apt-get install -y -qq sqlite3
    - gem install bundler --no-document
    - bundle install --jobs $(nproc) "${FLAGS[@]}"
```

`.gitlab-ci.yml`:

```yaml
include: 'templates/.before-script-template.yml'

rspec1:
  script:
    - bundle exec rspec
rspec2:
  script:
    - bundle exec rspec
```

Что делает `default:`

Задаёт настройки для **всех** jobs в pipeline. Каждый job получает их автоматически без явного указания.

Доступные ключи

`image` · `services` · `tags` · `before_script` · `after_script` · `cache` · `artifacts` · `retry` · `timeout` · `interruptible`

Override per-job

```yaml
rspec1:
  before_script: []   # отключить default
  script:
    - bundle exec rspec
```

Хороший паттерн: вынести в `default` всё, что повторяется — auth в registry, общие зависимости, retry-политику.

GitLab Cache — назначение

Cache хранит зависимости и переиспользуемые файлы между pipeline runs для ускорения.

Key Characteristics:

- **Scope**: разделяется между разными jobs и pipeline runs
- **Expiration**: можно задать TTL, после которого кэш инвалидируется
- **Key**: разные кэши для разных веток / jobs
- **Performance**: значительно ускоряет сборку

GitLab Cache — пример

```yaml
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .m2/repository/
```

Отдельный кэш на каждую ветку.

Инвалидируется автоматически при смене ветки.

Key стратегии

Key	Поведение
`${CI_COMMIT_REF_SLUG}`	кэш на ветку *(рекомендуется)*
`"deps-v1"`	общий для всех веток
`files: [package-lock.json]`	инвалидация по lock-файлу

Policy

- `pull-push` *(default)* — читать и обновлять
- `pull` — только читать *(test jobs)*
- `push` — только записывать *(build job)*

GitLab Artifacts — назначение

Artifacts хранят файлы, сгенерированные в job, для передачи в последующие jobs в том же pipeline.

Key Characteristics:

- **Scope**: специфичны для одного pipeline run
- **Expiration**: задаётся expire_in, после удаляются
- **Dependencies**: используются как зависимости для последующих jobs
- **Downloadable**: можно скачать из GitLab Web UI

GitLab Artifacts — назначение

Artifacts — файлы, сгенерированные job-ом и переданные в следующие jobs того же pipeline.

Свойство	Описание
Scope	один pipeline run
Expiration	задаётся `expire_in`
Dependencies	зависимости между jobs
Downloadable	доступны из GitLab Web UI

Типичные сценарии

- `build` собрал бинарник → `test` его использует
- Test job сгенерировал JUnit/SARIF отчёт → показывается в MR
- `terraform plan` сохранён → следующий job применяет

Срок жизни — выбирать осознанно

- `1 day` — для отладочных артефактов
- `1 month` — для релизных бинарников
- `never` — не удалять (расходует дисковое место!)

Скачать: UI → Job → Job artifacts → Browse/Download

Cache vs Artifacts — основные отличия

1. **Scope**:
   - **Cache**: между pipeline runs
   - **Artifacts**: внутри одного pipeline run
2. **Purpose**:
   - **Cache**: ускорение через переиспользование зависимостей
   - **Artifacts**: передача результатов работы между jobs

Cache vs Artifacts — expiration и keys

3. **Expiration**:
    - **Cache**: настраивается expire, но обычно живёт между pipelines
    - **Artifacts**: настраивается expire, обычно short-lived
4. **Key**:
    - **Cache**: keyed по строке — разные кэши для разных веток
    - **Artifacts**: без keys, привязаны к конкретному pipeline / job

Когда использовать Cache vs Artifacts

- **Use Cache**:
  - Зависимости и файлы переиспользуются между pipeline runs
  - Ускорение задач типа dependency installation
- **Use Artifacts**:
  - Передача файлов от одного job к следующим в том же pipeline
  - Хранение build outputs, test results, артефактов для дебага

Part 8: Реальные паттерны GitLab CI

5 инженерных задач и решения через pipeline logic

5 проблем — 5 паттернов

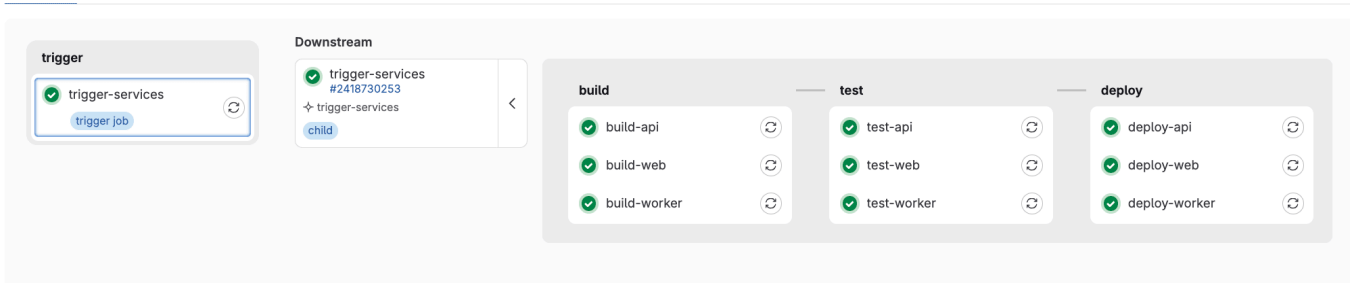
#	Проблема	Паттерн
1	Monorepo: каждый коммит запускает full rebuild	**Parent-Child Pipelines + DAG**
2	Микросервисы: breakage между репами видно только в проде	**Multi-Project Pipelines**
3	Multi-tenant: drift конфигурации между средами	**Dynamic Child Pipelines**
4	Тесты на feature branch ≠ тесты после merge	**MR Pipelines + Merged Results**
5	Platform team — бутылочное горлышко для standards	**CI/CD Components + Catalog**

Pattern 1 — Monorepo и full rebuild

Проблема: изменили README → запустилась сборка всех 20 сервисов на 45 минут.

Решение: Parent-Child Pipelines + DAG execution через `needs:`

- Каждый сервис в monorepo → свой child pipeline
- DAG ломает sequential ordering — параллелизм там, где раньше было ожидание
- Pipelines изолированы — фейл frontend не блокирует backend



Pattern 1 — конфигурация parent

```yaml
# .gitlab-ci.yml (parent)
stages:
  - trigger

trigger-services:
  stage: trigger
  trigger:
    include:
      - local: '.gitlab/ci/api-service.yml'   # child pipeline
      - local: '.gitlab/ci/web-service.yml'
      - local: '.gitlab/ci/worker-service.yml'
    strategy: depend   # parent ждёт все child
```

Что делает этот конфиг

- Parent pipeline — минимальный: одна задача запускает все child pipelines
- `strategy: depend` — без него parent сразу отмечается success, не дождавшись child
- `trigger:include` — child в том же репо и том же коммите (не путать с `trigger:project`)
- Добавьте `rules: changes: ['services/api/**']` — child запустится только при изменении своей папки
- Результат: rebuild только затронутых сервисов вместо полного rebuild monorepo

Pattern 1 — конфигурация child

```yaml
# .gitlab/ci/api-service.yml (child)
stages:
  - build
  - test

build-api:
  stage: build
  script:
    - echo "Building API service"

test-api:
  stage: test
  needs: [build-api]   # DAG: не ждёт весь build stage
  script:
    - echo "Running API tests"
```

Что добавляет child pipeline

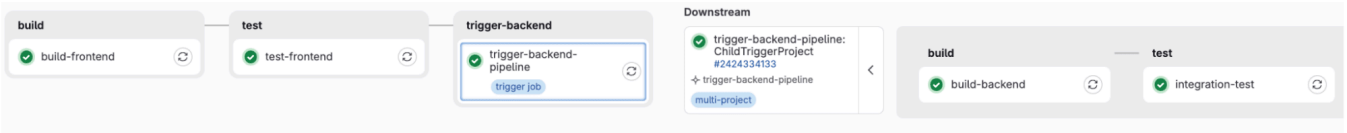
- Каждый сервис — независимый pipeline со своими stages и jobs
- `needs: [build-api]` — DAG: test-api стартует сразу после build-api, не дожидаясь остальных jobs в stage
- На одном сервисе незаметно, но на 10+ jobs DAG сокращает runtime в 2–3 раза
- В UI каждый child отображается отдельно — сразу видно, какой сервис упал
- Фейл одного child не блокирует остальные child pipelines

Pattern 2 — Microservices: cross-repo breakage

Проблема: frontend изменил API contract — backend узнаёт об этом в проде.

Решение: Multi-Project Pipelines — frontend pipeline триггерит backend pipeline и ждёт результата.

- Все pipelines связаны в UI — visibility между репами
- Backend получает API contract как artifact от frontend
- Integration test ловит breaking changes ДО merge



Pattern 2 — Frontend repo

```yaml
build-frontend:
  stage: build
  script:
    # генерирует API contract из OpenAPI spec
    - echo '{"api_version":"v2","breaking_changes":false}' \
        > dist/api-contract.json
  artifacts:
    paths: [dist/api-contract.json]   # передаётся в backend
    expire_in: 1 hour

trigger-backend-pipeline:
  stage: trigger-backend
  trigger:
    project: my-org/backend-service   # другой репо
    branch: main
    strategy: depend   # ждём backend; fail = MR блокирован
  rules:
    - if: $CI_COMMIT_BRANCH == "main"  # только на main
```

Что делает frontend pipeline

- Build генерирует `api-contract.json` — снимок контракта (версия, breaking changes)
- Artifact автоматически доступен backend pipeline через GitLab API
- `trigger:project` — запускает pipeline в другом репо (в отличие от `trigger:include`)
- `strategy: depend` — frontend MR блокируется при падении backend integration test
- `rules: main only` — на feature-branch frontend тестируется изолированно, backend не затрагивается
- В продакшне `api-contract.json` генерируется через `openapi-diff` для точного детекта breaking changes

Pattern 2 — Backend repo

```
integration-test:
  stage: test
  rules:
    # выполняется только по trigger, не по push
    - if: $CI_PIPELINE_SOURCE == "pipeline"
  script:
    # скачиваем artifact от frontend через GitLab API
    - curl --silent --fail
        --header "JOB-TOKEN: $CI_JOB_TOKEN"
        --output api-contract.json
        "${CI_API_V4_URL}/projects/${FRONTEND_PROJECT_ID}\
/jobs/artifacts/main/raw/dist/api-contract.json?job=build-frontend"
    - |
      if grep -q '"breaking_changes": true' api-contract.json; then
        echo "FAIL: Breaking API changes detected!"; exit 1
      fi
      echo "PASS: API contract compatible!"
```

Что делает backend pipeline

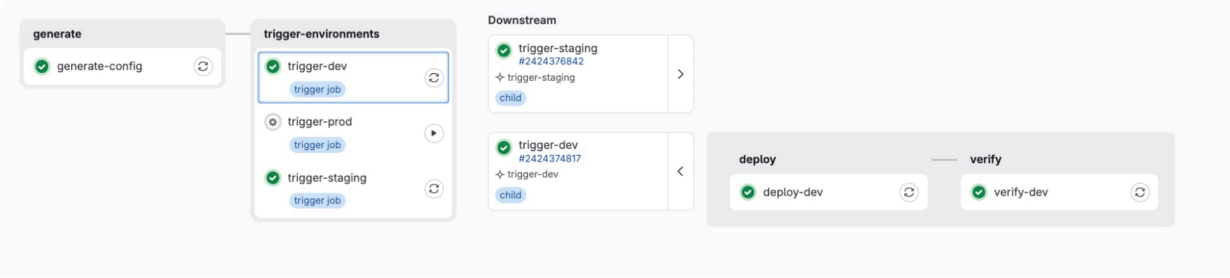
- `$CI_PIPELINE_SOURCE == "pipeline"` — job включается только при cross-project trigger, не при обычном push
- `$CI_JOB_TOKEN` — авторизация в GitLab API; токен ограничен текущим pipeline и автоматически создаётся
- `curl .../jobs/artifacts/.../raw/...` — скачивает artifact из frontend репо без дополнительных секретов
- `grep breaking_changes` — простая проверка контракта; `exit 1` роняет весь cross-project chain
- Целевой проект должен разрешить trigger: Settings → CI/CD → Token Access
- Итог: команда узнаёт о проблеме за 5 минут вместо инцидента в проде

Pattern 3 — Multi-tenant / Matrix deployments

Проблема: одни и те же шаги для 5 окружений (dev / qa / staging / preprod / prod) — drift конфигурации неизбежен.

Решение: Dynamic Child Pipelines — генерируем YAML на лету.

- Pipeline structure — это данные, не статика
- Один шаблон → N идентичных окружений
- Promotional gates (manual approval) встроены



Pattern 3 — генерация и trigger

```yaml
generate-config:
  stage: generate
  script:
    - |
      ENVIRONMENTS=${ENVIRONMENTS:-"dev staging prod"}
      for ENV in $ENVIRONMENTS; do
        cat > ${ENV}-pipeline.yml << EOF
      stages:
        - deploy
        - verify
      deploy-${ENV}:
        stage: deploy
        script:
          - echo "Deploying to ${ENV}"
      verify-${ENV}:
        stage: verify
        script:
          - echo "Smoke tests on ${ENV}"
      EOF
      done
  artifacts:
    paths:
      - "*.yml"
    exclude:
      - ".gitlab-ci.yml" # не тащим .gitlab-ci.yml в артефакт
```

Что происходит

- Job на лету генерирует отдельный `<env>-pipeline.yml` для каждого окружения
- Файлы сохраняются как artifacts — downstream trigger их подхватит

Почему важно

- Новая среда = одна строка в списке, без копирования блоков
- Логика deploy/verify централизована в генераторе

Gotchas

- `exclude: .gitlab-ci.yml` — маска `*.yml` иначе захватит сам `.gitlab-ci.yml`; чище генерировать в отдельную папку (`generated/*.yml`)
- В реальных проектах генератор сложнее: Jinja2 / envsubst / Helm templating
- Альтернатива: `include:component` из CI/CD Catalog (Pattern 5)

Pattern 3 — promotional gates

```yaml
.trigger-template:
  stage: trigger-environments
  trigger:
    strategy: depend # блокирует до завершения child

trigger-dev:
  extends: .trigger-template
  trigger:
    include:
      - artifact: dev-pipeline.yml
        job: generate-config

trigger-staging:
  extends: .trigger-template
  needs: [trigger-dev]  # только после dev
  trigger:
    include:
      - artifact: staging-pipeline.yml
        job: generate-config

trigger-prod:
  extends: .trigger-template
  needs: [trigger-staging] # только после staging
  trigger:
    include:
      - artifact: prod-pipeline.yml
        job: generate-config
  when: manual  # кнопка для релиз-менеджера
```

Promotion chain

- `needs:` строит цепочку dev → staging → prod: каждый следующий trigger стартует только при успехе предыдущего
- `when: manual` на prod — человек нажимает кнопку в UI

Dynamic Child Pipeline

- `include: artifact: ... job:` — динамический include из артефакта другого job, именно это и называется Dynamic Child Pipeline

Дополнения для prod

- `environment:` + protected environment + approval policies (Premium)
- `resource_group: production` — только один deploy в prod одновременно

Импакт: новая среда = одна строка в списке. Нет конфигурационного drift'a.

Pattern 4 — MR-first delivery

Проблема: тесты прошли на feature branch, после merge код собирается с main → конфликт ломает прод.

Решение: Merge Request Pipelines + Merged Results + Workflow Rules.

- На feature branch — только быстрые проверки
- На MR — полный suite на МЕРДЖЕННОМ результате
- На main — только deploy gate
- По расписанию — comprehensive scans

Pattern 4 — workflow rules

```yaml
workflow:
  rules:
    # MR event → pipeline запускается
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    # branch + открытый MR → НЕ создавать pipeline
    - if: $CI_COMMIT_BRANCH && $CI_OPEN_MERGE_REQUESTS
      when: never
    # просто branch без MR → запускается
    - if: $CI_COMMIT_BRANCH
    # schedule → запускается
    - if: $CI_PIPELINE_SOURCE == "schedule"

stages:
  - fast-checks
  - expensive-tests
  - deploy
```

Проблема без `workflow:`

- Push в branch с открытым MR создаёт **два** pipeline одновременно — branch-pipeline и MR-pipeline
- Удвоенная нагрузка на runners, удвоенный расход CI-минут

Что делает этот `workflow:`

- Правило 2 (`when: never`) отсекает дублирующий branch-pipeline, если уже есть MR
- Итог: **один** pipeline на каждое событие

Порядок имеет значение

- Rules вычисляются сверху вниз, первое совпадение побеждает
- `when: never` должно стоять **до** общего `$CI_COMMIT_BRANCH`

Pattern 4 — fast checks vs expensive

```yaml
lint-code:
  stage: fast-checks
  rules:
    - if: $CI_PIPELINE_SOURCE == "push"
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"
    # везде — секунды выполнения

integration-tests:
  stage: expensive-tests
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"
    # ~15 минут — только перед влитием

e2e-tests:
  stage: expensive-tests
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"
    # ~30 минут — только перед влитием
```

Стратегия «right tests at right time»

- `lint` — секунды → запускать везде, обратная связь мгновенная
- `integration-tests` — 15 мин → только MR и main
- `e2e-tests` — 30 мин → только MR и main

Почему не гонять e2e на feature branch

- Feature branch без MR — черновик разработчика, код ещё не идёт в общую базу
- Когда MR открыт — разработчик готов к ревью, CI-ресурсы оправданы

Экономия

- 50 feature branch pushes/день × 30 мин e2e = ~25 CI-часов сэкономлено
- Экономия растёт пропорционально команде

Pattern 4 — nightly scan + prod gate

```yaml
nightly-comprehensive-scan:
  stage: expensive-tests
  script:
    - echo "Running full nightly suite (2 hours)"
  rules:
    # только schedule-триггер
    - if: $CI_PIPELINE_SOURCE == "schedule"
    # UI: Build → Pipeline schedules → cron 0 2 * * *

deploy-production:
  stage: deploy
  script:
    - echo "Deploying to production"
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual  # кнопка в UI
    # ужесточение: if: $CI_COMMIT_TAG
    # when: manual  → только теги релизов
```

Nightly scan

- Тяжёлые проверки (full e2e, performance, deep security) — раз в сутки, не мешают дневным pipeline
- Настройка: Build → Pipeline schedules → `0 2 * * *`

Prod gate

- `when: manual` на main — релиз-менеджер нажимает кнопку осознанно
- Ужесточение: `if: $CI_COMMIT_TAG` — только теги релизов попадают в prod

Итог всей стратегии Pattern 4

- Один `.gitlab-ci.yml`, три разных сценария: push → lint, MR → lint+tests, schedule → полный scan
- Максимальная защита main/prod при минимальных CI-затратах

Pattern 4 — pipeline по контексту

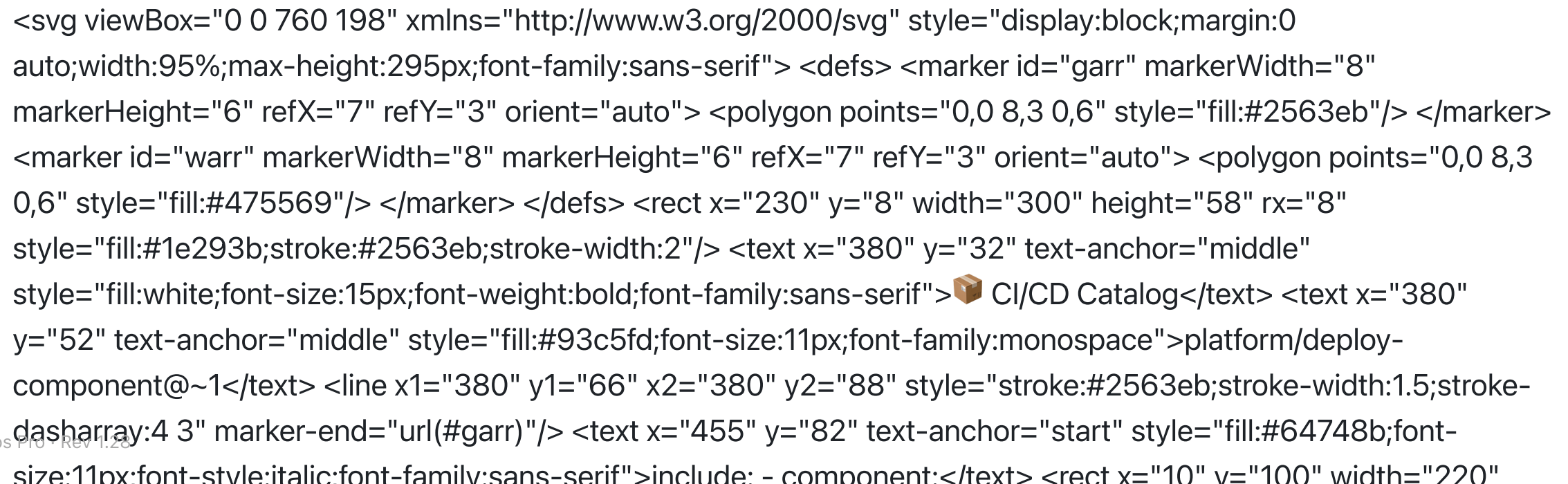


Pattern 5 — Governed pipelines

Проблема: Platform team хочет стандарт `deploy` для 50 команд, но обновление = ручной обход 50 репозиториев.

Решение: CI/CD Components — версионируемые pipeline-блоки в CI/CD Catalog.

- Один компонент = один источник правды · версии через Git tags
- `include: - component:` в любом проекте · Compliance "by default"



Pattern 5 — определение компонента

```yaml
# templates/deploy.yml
spec:
  inputs:
    stage:
      default: deploy
    environment:
      default: production
---
deploy-job:
  stage: $[[ inputs.stage ]]
  script:
    - echo "Deploying $APP_NAME to $[[ inputs.environment ]]"
    - echo "Deploy URL: $DEPLOY_URL"
  environment:
    name: $[[ inputs.environment ]]
```

Pattern 5 — использование компонента

```yaml
# .gitlab-ci.yml в application repo
variables:
  APP_NAME: "my-awesome-app"
  DEPLOY_URL: "https://api.example.com"

include:
  # формат: gitlab.com/<org>/<library>/<component>@<version>
  - component: gitlab.com/my-org/component-library/build@1.0
  - component: gitlab.com/my-org/component-library/test@1.0
  - component: gitlab.com/my-org/component-library/deploy@1.0
    inputs:
      environment: staging  # пробрасывается в spec:inputs:

stages:
  - build
  - test
  - deploy
```

Версионирование компонентов

- `@1.0.0` — точный semver, стабильно для prod
- `@1` — последний релиз `1.x.x`; `@1.0` — последний patch `1.0.x`
- `@main` — ветка, только для разработки
- SHA — точный коммит, максимальный контроль

`inputs:` — конфигурация без копирования кода

- Значения попадают в `spec:inputs:` компонента
- Каждая команда задаёт своё окружение, логика остаётся в компоненте

Real-world применение

- Стандартный Helm deploy — один компонент для всех команд
- Security scan suite — компонент Platform team, встраивается везде
- Compliance reporting — обязательный компонент через Compliance Framework

Pattern 5 — важные нюансы компонентов

`$[[ inputs.X ]]` — parse time

```
# ✅ Работает — используем variables:
variables:
  MY_VAR: $[[ inputs.name ]]
script:
  - echo "$MY_VAR"   # shell runtime

# ⚠ Работает, но хрупко — текстовая вставка:
script:
  - echo "$[[ inputs.name ]]"  # кавычка во входе ломает shell
```

Execution context

```
component-library/
  templates/template.yml  ← описание job'а
  src/script.py           ← НЕДОСТУПЕН у потребителя

consumer-repo/            ← вот где job выполняется
  .gitlab-ci.yml
  # src/script.py нет!
```

Файлы компонента **не копируются** к потребителю. Весь исполняемый код должен быть встроен в шаблон или взят из образа Docker.

Что даёт композиция паттернов

Технически

- Pipeline runtime ↓ за счёт DAG/parallel
- Cross-service breakage ловится в CI
- Drift конфигурации устранён
- CI costs ↓ через context-aware tests

Организационно

- Platform team не bottleneck
- Compliance — default, не отдельный процесс
- Pipeline отражает реальную структуру команд
- Visibility на каждом уровне

> 5 паттернов — это **строительные блоки**. Composes them — и получите систему доставки, отражающую вашу организацию.

🛠️ Time to make hands dirty — Lab 2

Monorepo CI/CD

Conditional CI: только затронутые сервисы запускаются при изменениях в монорепо.

📂 **labs/gitlab_ci_lab_monorepo.md**

🛠️ Time to make hands dirty — Lab 3

Первый CI/CD Component

Создаём переиспользуемый компонент со `spec:inputs:` и подключаем из другого проекта.

📂 **labs/gitlab_ci_lab_component.md**

Part 9: Best Practices

Чек-лист зрелого pipeline

GitLab CI/CD Best Practices

- **Use caches and artifacts effectively** — ускоряем pipeline
- **Read and share documentation** — официальные docs регулярно обновляются
- **Optimize pipeline stages** — DAG / parallel
- **Leverage failures for improvement** — учимся на падениях
- **Test environments should mirror production** — production parity

Use Caches and Artifacts Effectively

- **Cache**: сохраняем/загружаем файлы один раз, переиспользуем
- **Artifacts**: передаём файлы между jobs в том же pipeline
- **Tag runners** — консистентные теги для jobs, использующих кэши
- **Use unique cache keys** — для изоляции workflows (per branch, per platform)

Read the Documentation

- GitLab documentation is extensive and regularly updated
- Encourage team members to review docs and FAQs
- Bookmark important sections for quick reference

Optimize Pipeline Stages

- **Organize jobs** — структурировать так, чтобы фейлы было легко находить
- **Run jobs as early as possible** — fail fast
- **Avoid grouping similar jobs** — если это замедляет pipeline

Use Failures to Improve Processes

- **Foster a culture of learning, not blame** — без виноватых
- **Investigate root causes of failures** — RCA после каждого фейла
- **Use `allow_failure`** — для non-critical jobs
- **Frequent commits make issues easier to trace** — bisect-friendly

Test Environments Should Mirror Production

- **Match test and production** — одинаковая конфигурация
- **Use the "test pyramid"** — много unit, мало e2e
- **Leverage GitLab Review Apps** — preview production-like окружений

Pipeline как quality gates

Каждая стадия — это автоматический gate. Прошёл — едешь дальше. Упал — следующие не запускаются.

Stages = gates:

- **build** — компиляция, артефакты собрались
- **test** — unit + integration зелёные
- **security** — SAST/DAST/SCA не нашли critical
- **deploy** — манифесты применились

Что даёт это мышление:

- сломанный код **не доходит** до main
- сломанный артефакт **не уходит** в prod
- регрессия видна **в момент появления**, а не от пользователя

CD best practices — 6 правил

- **Start where you are; iterate progressively** — не пытаемся «сделать CD за квартал»
- **Minimize tool complexity** — меньше инструментов → меньше моментов отказа
- **Track issues systematically** — каждый incident → ticket → post-mortem
- **Automate user acceptance testing** — UAT на dev/staging без участия людей
- **Manage releases through automation** — версионирование, changelog, теги — из CI
- **Implement monitoring** — без observability нет CD

CD best practices — 6 правил

- **Start where you are; iterate progressively** — не пытаемся «сделать CD за квартал»
- **Minimize tool complexity** — меньше инструментов → меньше моментов отказа
- **Track issues systematically** — каждый incident → ticket → post-mortem
- **Automate user acceptance testing** — UAT на dev/staging без участия людей
- **Manage releases through automation** — версионирование, changelog, теги — из CI
- **Implement monitoring** — без observability нет CD

CI = собрали и протестировали. CD = доставили до пользователя.

- Итеративный старт: автоматизируй то, что уже делаешь руками — один шаг за раз
- Меньше инструментов: GitLab + один runner + один registry > зоопарк из 15 инструментов
- Системный трекинг: без Issue на каждый pipeline failure повторяющиеся проблемы живут вечно
- UAT автоматом: Cypress / Playwright / Robot Framework делают то, что QA делал руками
- Релизы из CI: semantic-release, auto-changelog, git tags — не руками
- Мониторинг обязателен: deployment markers в Grafana → видна корреляция deploy → metric spike

Миграция в GitLab CI/CD

Откуда чаще всего мигрируют:

Source	Common pain	GitLab CI/CD путь
Jenkins	groovy, plugins, отдельный сервер	`.gitlab-ci.yml`, builtin runners, CI/CD Catalog
Bamboo	Atlassian lock-in, EOL	YAML pipelines, GitLab Issues + MRs
GitHub Actions	matrix limits, marketplace risk	typed `inputs`, `parallel:matrix`, components
CircleCI	orbs, lim. self-hosted	reusable templates, own runners

Стратегия:

1. Mirror репозиторий → GitLab (read-only)
2. Параллельно пишем `.gitlab-ci.yml`, оставляем legacy CI работающим
3. Переключаем команды постепенно — pilot project → весь monorepo
4. После 2-3 спринтов стабильности → выключаем legacy

AI — Root Cause Analysis

Проблема: Pipeline упал на 1500-строчном логе. Кто и за сколько минут найдёт причину?

GitLab Duo Root Cause Analysis:

- читает full job log,
- видит failed step и контекст,
- предлагает hypothesis: «вероятная причина — X»,
- ссылается на specific log line,
- предлагает fix: код или конфигурацию.

> 🎯 ROI: переход от «новичок ищет 30 минут» к «AI даёт направление за 30 секунд»

CI/CD Steps — next-gen DSL

Эксперимент GitLab: новый язык для CI/CD automation, поверх YAML.

Идея: YAML был хорош, пока pipeline — это «5 stages × 10 jobs». Дальше начинаются проблемы:

- логика в `script:` через bash, всегда,
- условия в `rules:` многословны и фрагментированы,
- переиспользование через include + extends — boilerplate.

Steps approach:

- structured composable units,
- typed inputs/outputs (как functions),
- runs anywhere — GitLab Runner или standalone,
- не привязан к YAML editor — есть programmatic API.

> ⚠️ Steps — в активной разработке, не production-ready на 17.x. Следим за релизами.

🛠️ Time to make hands dirty — Lab 4

Terraform через GitLab CI

GitLab Managed State: plan → manual apply → manual destroy без `terraform apply` с ноутбука.

📂 **labs/gitlab_ci_lab_tf.md**

🛠️ Time to make hands dirty — Lab 5

Security scan с Trivy

Image scan через DinD + отчёт в Security tab + repo scan для исходников.

📂 **labs/gitlab_ci_lab_basic_trivy_scan.md**

🛠️ Time to make hands dirty — Lab 6

Подпись образов с Cosign

Keyless подпись через OIDC id_tokens, верификация в Rekor transparency log.

📂 **labs/gitlab_ci_lab_cosign.md**

🛠️ Time to make hands dirty — Lab 7

Five environments через child pipelines

Parent → child → grand-child pipelines: review, integration, staging, qa, production.

📂 **labs/gitlab_ci_lab_five_envs.md**

Конец курса

Что узнали

- Основы CI/CD и continuous-методологии
- Архитектура и компоненты GitLab CI/CD
- Структура `.gitlab-ci.yml` от простого к сложному
- 5 паттернов решения реальных инженерных задач
- Best practices для зрелого pipeline

Следующий шаг

- Соберите свой первый pipeline на реальном проекте
- Внедрите ОДИН из паттернов: monorepo / MR-first / Components
- Добавьте security scans (Trivy, SAST)

Questions?

References

- GitLab CI/CD Documentation
- `.gitlab-ci.yml` keyword reference
- Predefined CI/CD variables
- GitLab CI/CD examples
- DORA metrics — Google Cloud
- The CI/CD Pipeline — Octopus
- CI/CD Pipeline Explained — Softjourn
- CI/CD in Cloud Computing — Civo